

Contents

Cost-Aware Routing for Web Usage Agents: Literature Review	2
Executive Summary	2
Table of Contents	3
1. Introduction	3
2. Individual Paper Analysis	3
Paper 1: AgentSwing	3
Paper 2: Read More, Think More	4
Paper 3: M ² (Dual-Memory Augmentation)	5
Paper 4: WebRouter	7
Paper 5: Dual-Modality Multi-Stage Adversarial Safety Training (DMAST)	8
Paper 6: Hierarchical Memory Tree (HMT)	9
Paper 7: Adaptive VLM Routing (AVR)	10
Paper 8: Agentic Test-Time Scaling (CATTS)	11
Paper 9: WebWorld	12
Paper 10: ContractSkill	13
Paper 11: OpAgent	14
Paper 12: SkillWeaver	16
Paper 13: StructuredAgent	17
Paper 14: Web-CogReasoner	18
Paper 15: WEBSERV	19
Paper 16: MoE-ICRL (Mixture-of-Experts Meets In-Context Reinforcement Learning)	20
Paper 17: Ego2Web	21
Paper 18: Optimizing Generative AI Networking	22
Paper 19: Egocentric Co-Pilot	23
Paper 20: Throttling Web Agents Using Reasoning Gates	24
Paper 21: GUIDE	25
Paper 22: WebGraphEval	26
Paper 23: WebCanvas	27
Paper 24: MoMA (Mixture of Models and Agents) / Generalized Routing	28
3. Cross-Paper Synthesis	30
Dimension 1: Observation Representation	30
Dimension 2: Cost-Aware Design	31
Dimension 3: Routing/Model Selection	31
Dimension 4: Memory Management	32
Dimension 5: Grounding/Failure Recovery	33
Dimension 6: Small Models $\leq 10B$	34
Dimension 7: Benchmark Methodology	35
Dimension 8: Task Planning/Decomposition	35
4. Gaps in Literature	36
Gap 1: Step-Level Adaptive Routing for Small VLMs on Complex Benchmarks	36
Gap 2: Interaction Between Observation Modality and Failure Recovery for VLMs	37
Gap 3: Memory Management for Small VLMs on Long-Horizon Visual Tasks	37

Gap 4: Cost-Aware Module Ablation for Web Agents	38
Gap 5: Benchmark-Specific Routing Strategies	38
Gap 6: Small Model Confidence Calibration for Routing	38
Gap 7: Real-World Deployment Considerations	39
5. Priority Recommendations for P79	39
Recommendation 1: Prioritize SoM Representation for Qwen3-VL-4B (Phase 1)	39
Recommendation 2: Implement Three-Mechanism Routing (Phase 2) .	40
Recommendation 3: Implement Dual-Tier Hierarchical Memory (Module M4)	41
Recommendation 4: Implement Adaptive Retry with Fault Localization (Modules M1 & M2)	42
Recommendation 5: Implement Knowledge-Driven Two-Stage Reasoning (Module M3)	43
6. Conclusion	44
7. References	45

Cost-Aware Routing for Web Usage Agents: Literature Review

Project P79

Date: April 7, 2026

Executive Summary

This literature review analyzes 24 recent papers (2024-2026) on web agents, focusing on dimensions critical to Project P79’s investigation of cost-aware routing for web usage agents. The review systematically examines each paper across eight research dimensions: Observation Representation, Cost-Aware Design, Routing/Model Selection, Memory Management, Grounding/Failure Recovery, Small Models $\leq 10B$, Benchmark Methodology, and Task Planning/Decomposition.

Key findings reveal that while cost-aware routing has emerged as a critical research direction, most approaches focus on query-level routing rather than step-level adaptive routing during task execution. The literature demonstrates strong consensus on the value of accessibility tree (A11y) representations for smaller models, the effectiveness of hierarchical memory structures for long-horizon tasks, and the importance of confidence-based routing mechanisms. However, significant gaps remain in understanding optimal routing strategies for small vision-language models ($\leq 10B$ parameters) on complex benchmarks like VisualWebArena, particularly regarding the interaction between observation modality (DOM/SoM/hybrid), memory management, and failure recovery mechanisms.

For P79’s experimental design using Qwen3-VL-4B on VisualWebArena, the literature provides actionable guidance on representation comparison (Phase 1), rule-based

routing strategies (Phase 2), and module ablation (Phase 3), with specific recommendations for retry mechanisms, fallback strategies, two-stage reasoning, and memory management approaches.

Table of Contents

1. [Introduction](#)
 2. [Individual Paper Analysis](#)
 - Papers 1-24 (detailed analysis)
 3. [Cross-Paper Synthesis](#)
 4. [Gaps in Literature](#)
 5. [Priority Recommendations for P79](#)
 6. [Conclusion](#)
 7. [References](#)
-

1. Introduction

Project P79 investigates cost-aware routing strategies for web usage agents, specifically examining how to optimize the trade-off between task success and computational cost when deploying small vision-language models on complex web navigation benchmarks. This literature review systematically analyzes 24 papers published between 2024 and 2026, providing a comprehensive foundation for P79’s experimental design.

The review is structured to provide both depth and actionability. Each paper receives detailed analysis covering its core contribution, quantitative results, architectural decisions, and limitations. Following the individual analyses, we synthesize findings across papers to identify consensus, contradictions, and gaps in the current literature. Finally, we provide prioritized recommendations specifically tailored to P79’s three-phase experimental design: Phase 1 (representation comparison), Phase 2 (rule-based routing), and Phase 3 (module ablation with M1=retry, M2=fallback, M3=two-stage reasoning, M4=memory management).

Two papers—Avenir-Web (arXiv:2602.02468) and Chain-of-Ground (arXiv:2512.01979)—are referenced for comparison where relevant but are not included as standalone analyses, as they were already available to the P79 team.

2. Individual Paper Analysis

Paper 1: AgentSwing

Citation: Zhaopeng Feng, Liangcai Su, Zhen Zhang, Xinyu Wang, Xiaotian Zhang, Xiaobin Wang, Runnan Fang, Qi Zhang, Baixuan Li, Shihao Cai, Rui Ye, Hui Chen, Jiang Yong, Joey Tianyi Zhou, Chenxiong Qian, Pengjun Xie, Bryan Hooi, Zuozhu Liu, Jingren Zhou. “AGENTSWING: Adaptive Parallel Context Management Routing for Long-Horizon Web Agents.” arXiv:2603.27490v1 [cs.CL], 2026 [1].

Core Contribution: AgentSwing introduces an adaptive parallel context management routing framework that expands multiple context-managed branches (Discard-All, Keep-Last-N, Summary) in parallel at each trigger point and uses lookahead routing to select the most promising continuation. Unlike static context management methods that commit to a single strategy throughout the trajectory, AgentSwing dynamically selects among strategies based on short-horizon lookahead (K additional turns) to balance search efficiency and terminal precision [1].

Key Quantitative Results: - **WebArena (GPT-OSS-120B):** AgentSwing achieved 60.0% success rate vs. baseline 39.5%, Discard-All 50.5%, Keep-Last-N 52.5%, Summary 48.0% [1] - **WebArena (DeepSeek-v3.2):** 62.5% vs. baseline 51.4% [1] - **Efficiency Metrics:** AgentSwing achieved 56.7% success with 190.3k tokens, compared to Keep-Last-N’s 47.3% success with 205.4k tokens [1] - **Lookahead Ablation:** k=3 achieved 60.0% (optimal), k=1 achieved 52.5%, k=5 achieved 55.0%, showing moderate lookahead is most effective [1]

Relevant Dimensions: - **(3) Routing/Model Selection:** Core contribution—adaptive routing among context management strategies - **(4) Memory Management:** Core contribution—parallel context management with three candidate strategies - **(2) Cost-Aware Design:** Addresses search efficiency and terminal precision as cost-aware dimensions

Actionable Insights for P79: - **Phase 2 (Rule-based Routing):** AgentSwing’s lookahead mechanism (k=3 optimal) provides a concrete template for P79’s routing logic. Consider implementing confidence-based triggers that expand multiple branches when uncertainty is high, then select based on short-horizon evaluation. - **Module M4 (Memory Management):** The parallel evaluation of Discard-All, Keep-Last-N, and Summary strategies directly informs P79’s memory module design. For Qwen3-VL-4B with limited context, test these three strategies with trigger points at 70-80% of max context length. - **Phase 3 (Module Ablation):** AgentSwing demonstrates that combining multiple strategies yields complementary advantages. Test M4 with and without adaptive selection to measure marginal contribution. - **Visual-WebArena Application:** AgentSwing’s 3x reduction in interaction turns while maintaining performance suggests that adaptive memory management could significantly reduce token costs for P79’s small model deployment.

Limitations/Caveats: - Routing performed by the agent model itself may not be optimal; a dedicated router could improve branch selection [1] - Static strategies underperform at small turn budgets, limiting applicability to very short tasks [1] - Larger lookahead (k=5) doesn’t always improve performance, risking context length violations [1] - Experiments used models $\geq 30B$ parameters; effectiveness for Qwen3-VL-4B requires validation

Paper 2: Read More, Think More

Citation: Masafumi Enomoto, Ryoma Obara, Haochen Zhang, Masafumi Oyamada. “Read More, Think More: Revisiting Observation Reduction for Web Agents.” arXiv:2604.01535v1 [cs.CL], 2026 [2].

Core Contribution: This paper challenges the prevailing assumption that observation reduction universally benefits web agents, demonstrating that optimal observation representation depends on model capability and thinking token budget. Higher-capability models benefit from detailed HTML observations, while lower-capability models perform better with compact accessibility tree (a11y) representations. The study also highlights the importance of observation history, with diff-based representations offering a token-efficient alternative [2].

Key Quantitative Results: - **Higher-Capability Models (WorkArena L1):** gpt-5.1 with HTML achieved 73.3% success rate, 17.5pp improvement over a11y [2] - **Lower-Capability Models:** gpt-oss-120b with HTML decreased to 38.8%, 7.9pp worse than a11y; gpt-oss-20b with HTML decreased to 27.6%, 18.8pp worse than a11y [2] - **Observation History:** Adding 4 steps of history (hist4) improved gpt-5.1 by 3.0pp to 58.8%; improved gemini-2.5-flash by 10.9pp to 39.4% [2] - **Diff-based History:** Achieved comparable performance to full history while reducing input tokens to approximately one-third [2]

Relevant Dimensions: - **(1) Observation Representation:** Core focus—comparing HTML, a11y, and a11y+screenshots - **(2) Cost-Aware Design:** Addresses thinking token budget and token efficiency of diff-based history - **(4) Memory Management:** Investigates observation history (hist0, hist4, hist9, full, diff)

Actionable Insights for P79: - **Phase 1 (Representation Comparison):** This paper provides the most direct guidance for P79’s Phase 1. For Qwen3-VL-4B (a small model), expect a11y to outperform HTML. Test DOM (full HTML), SoM (set-of-marks, similar to a11y), and hybrid representations, predicting SoM will perform best. - **Module M4 (Memory Management):** Implement diff-based observation history to reduce token consumption. For VisualWebArena’s longer tasks, hist4 (4 steps) provides a good balance between context and efficiency. - **Phase 2 (Routing):** Consider routing rules that switch from a11y to HTML only when the model’s confidence is very high and the task requires fine-grained layout understanding (e.g., form filling with complex CSS). - **Benchmark Consideration:** Results are from WorkArena L1 (ServiceNow); generalization to VisualWebArena requires validation, but the principle (smaller models prefer compact representations) should hold.

Limitations/Caveats: - Experiments conducted only on WorkArena L1; generalizability to other websites/domains unverified [2] - Used id-based grounding; trends for coordinate-based grounding (used in VisualWebArena) may differ [2] - Mechanism of HTML benefit (CSS layout cues) not directly verified through ablation [2] - Effect of combining richer observations (HTML) with history unexplored; impact on longer-horizon tasks (>15 steps) unknown [2]

Paper 3: M² (Dual-Memory Augmentation)

Citation: Dawei Yan, Haokui Zhang, Guangda Huzhang, Yang Li, Yibo Wang, Qing-Guo Chen, Zhao Xu, Weihua Luo, Ying Li, Wei Dong, Chunhua Shen. “M²: Dual-Memory Augmentation for Long-Horizon Web Agents via Trajectory Summarization

and Insight Retrieval.” arXiv:2603.00503v1 [cs.CV], 2026 [3].

Core Contribution: M² proposes a training-free, memory-augmented framework with a dual-tier memory mechanism: Dynamic Trajectory Summarization (Internal Memory) compresses verbose interaction history into concise state updates, and Insight Retrieval Augmentation (External Memory) guides the agent with actionable guidelines from an offline insight bank. This approach decouples task performance from context growth, enabling high-fidelity navigation with sustainable computational costs [3].

Key Quantitative Results: - **WebVoyager (Qwen3-VL-32B):** M² achieved 74.0% accuracy (16.2% increase) with 58.7% token reduction (from 315.6k to 130.2k tokens) [3] - **WebVoyager (Claude-3.7-Sonnet):** 84.5% accuracy (12.5% increase) with 55.0% token reduction (from 300.3k to 135.2k tokens) [3] - **OnlineMind2Web (Qwen3-VL-32B):** Success rate increased from 31.96% to 51.55% (19.6% increase) with 58.7% token reduction [3] - **Token Consumption Growth:** On 16-step task, M² reduced total consumption from 106k to 58k tokens (45.3% reduction) [3] - **Insight Retrieval Latency:** Approximately 6 milliseconds per retrieval [3]

Relevant Dimensions: - **(1) Observation Representation:** Replaces raw observations with concise textual summaries - **(2) Cost-Aware Design:** Core focus—optimizing context efficiency and reducing token consumption - **(4) Memory Management:** Core contribution—dual-tier memory (Internal + External) - **(5) Grounding/Failure Recovery:** External Memory provides defensive hints to prevent historical failures - **(6) Small Models $\leq 10B$:** Enables Qwen3-VL-32B to match proprietary performance

Actionable Insights for P79: - **Module M4 (Memory Management):** M²’s dual-tier approach is highly relevant. Implement Internal Memory by prompting Qwen3-VL-4B to generate concise state summaries at each step (e.g., “Thought: X, Action: Y, Result: Z” $\rightarrow$ “Current state: On product page, price filter applied”). - **Module M2 (Fallback):** External Memory’s insight bank concept can inform P79’s fallback strategy. Pre-compute a small bank of successful VisualWebArena trajectories (e.g., 50-100 examples), retrieve top-5 similar cases via semantic similarity when the agent encounters low confidence. - **Phase 3 (Ablation):** Test M4 with Internal Memory only, External Memory only, and both combined to measure marginal contributions. M²’s results suggest Internal Memory is more critical (primary token savings), while External Memory provides defensive guidance. - **Cost-Aware Design:** M²’s 58.7% token reduction with Qwen3-VL-32B provides a concrete target for P79. Aim for 40-50% token reduction with Qwen3-VL-4B while maintaining or improving success rate.

Limitations/Caveats: - Initial overhead of $\sim 3.6k$ tokens at Step 1 due to Insight Retrieval Augmentation [3] - Insight Bank constructed from 55k successful trajectories; P79 may need to start with smaller bank (100-500 trajectories) due to VisualWebArena’s limited data - Effectiveness for models $< 10B$ parameters not directly tested; Qwen3-VL-4B may require more aggressive summarization - Baseline agents can suffer from “global search trap” and “contextual hallucination” that M² mitigates, but these failure modes need validation on VisualWebArena

Paper 4: WebRouter

Citation: Tao Li, Jinlong Hu, Yang Wang, Junfeng Liu, Xuejun Liu. “WEBROUTER: QUERY-SPECIFIC ROUTER VIA VARIATIONAL INFORMATION BOTTLENECK FOR COST-SENSITIVE WEB AGENT.” arXiv:2510.11221v1 [cs.CL], 2025 [4].

Core Contribution: WebRouter addresses the cost-performance trade-off by introducing a query-specific router trained from an information-theoretic perspective using a cost-aware Variational Information Bottleneck (ca-VIB) objective. This approach learns a compressed representation of the input prompt, explicitly penalizing expected operational cost while filtering out irrelevant information for robust routing decisions on noisy, high-dimensional web agent inputs [4].

Key Quantitative Results: - **Operational Cost Reduction:** 87.8% reduction compared to GPT-4o baseline (from \$0.98 to \$0.12 average price) with only 3.8% accuracy drop [4] - **WebVoyager Benchmark:** WebRouter achieved 82.3% average accuracy vs. RouterDC’s 67.8%, with average price of \$0.12 vs. GPT-4o’s \$1.01 [4] - **Prompt Token Contribution:** Prompt tokens contribute over 70% of total price for all models [4] - **Running Time:** Only 14% slower than GPT-4o baseline [4] - **Query Encoder:** mDeBERTaV3-base with 768-dimensional embedding space [4]

Relevant Dimensions: - **(2) Cost-Aware Design:** Core contribution—ca-VIB objective explicitly penalizes operational cost - **(3) Routing/Model Selection:** Core contribution—query-specific router for dynamic LLM matching - **(4) Memory Management:** Router input includes agent’s action history

Actionable Insights for P79: - **Phase 2 (Rule-based Routing):** WebRouter’s ca-VIB approach provides a principled framework for P79’s routing logic. While P79 uses rule-based routing (not learned), the insight that prompt tokens dominate cost (>70%) is critical. Design routing rules that minimize prompt length when routing to more expensive operations (e.g., two-stage reasoning). - **Module M3 (Two-stage Reasoning):** WebRouter’s 87.8% cost reduction suggests that selective application of expensive reasoning is highly effective. Implement M3 to trigger two-stage reasoning only when: (1) initial action confidence is low, (2) task complexity is high (detected via heuristics like form density), or (3) previous action failed. - **Routing Architecture:** WebRouter uses a lightweight encoder (mDeBERTaV3-base, 768-dim). For P79’s rule-based approach, consider using Qwen3-VL-4B’s internal confidence scores or a simple heuristic based on action history length and task progress. - **Cost Metrics:** Track prompt tokens separately from completion tokens to identify routing optimization opportunities.

Limitations/Caveats: - Paper doesn’t detail specific limitations, but implicitly acknowledges challenges with verbose prompts in web agent scenarios [4] - WebRouter is a learned router requiring training data; P79’s rule-based approach must approximate these learned patterns with heuristics - Experiments on WebVoyager (5 websites); generalization to VisualWebArena’s diverse task types requires validation - Query-level routing (one decision per task) differs from P79’s step-level routing needs

Paper 5: Dual-Modality Multi-Stage Adversarial Safety Training (DMAST)

Citation: Haoyu Liu, Dingcheng Li, Lukas Rutishauser, Zeyu Zheng. “Dual-Modality Multi-Stage Adversarial Safety Training: Robustifying Multimodal Web Agents Against Cross-Modal Attacks.” arXiv:2603.04364v1, 2026 [5].

Core Contribution: DMAST proposes a framework to harden multimodal web agents against adversarial co-evolution by formalizing agent-attacker interaction as a two-player zero-sum Markov game. It employs a three-stage pipeline: imitation learning from a strong teacher model, oracle-guided supervised fine-tuning with a novel zero-acknowledgment strategy, and adversarial reinforcement learning via Group Relative Policy Optimization (GRPO) self-play. This addresses coordinated cross-modal attacks that alter both screenshot and accessibility tree modalities [5].

Key Quantitative Results:

- **Vulnerability Analysis (MiniWob++, Gemma-3-27B-IT):** No Attack: 36.9% ASR; Text-Only Attack: 15.9% ASR, 24.1% Atk; Image-Only Attack: 15.6% ASR, 34.4% Atk; Dual Attack: 15.8% ASR, 35.7% Atk [5]
- **DMAST Performance (MiniWob++):** Base Model: 18.9% ASR, 14.0% TSR; DMAST Full: 10.8% ASR, 25.7% TSR; DMAST + Prompt Defense: 4.5% ASR, 25.0% TSR [5]
- **DMAST Performance (VisualWebArena):** Base Model: 41.2% ASR, 6.2% TSR; DMAST Full: 21.4% ASR, 10.2% TSR; DMAST + Prompt Defense: 7.2% ASR, 8.2% TSR [5]
- **Stage-wise Contributions:** Imitation Learning improved ASR to 16.8% (MiniWob++); Oracle-Guided SFT improved TSR to 23.3%; Adversarial RL yielded largest combined gain (10.8% ASR, 25.7% TSR) [5]

Relevant Dimensions:

- **(1) Observation Representation:** Uses both screenshots and accessibility trees (AX-Tree) as dual-modality observations
- **(6) Small Models $\leq 10B$:** Student model is Gemma-3-12B-IT
- **(7) Benchmark Methodology:** Systematic vulnerability analysis and adversarial evaluation framework

Actionable Insights for P79:

- **Phase 1 (Representation Comparison):** DMAST’s finding that image-only and dual attacks are more effective than text-only attacks (34.4% vs. 24.1% attacker success) suggests that visual modality is more vulnerable. For P79’s hybrid representation, ensure robust grounding between screenshot and DOM/SoM to prevent inconsistencies.
- **Module M1 (Retry):** DMAST’s adversarial RL stage demonstrates that agents can learn to recover from failures through self-play. While P79 doesn’t use RL, the insight that retry mechanisms should be adaptive (not fixed) is valuable. Implement M1 with escalating strategies: first retry with same representation, second retry with different representation (e.g., DOM→SoM), third retry with two-stage reasoning.
- **VisualWebArena Baseline:** DMAST reports 6.2% TSR for base Gemma-3-12B-IT on VisualWebArena, improving to 10.2% with full pipeline. This provides a reference point for P79’s Qwen3-VL-4B baseline expectations (likely 5-15% TSR).
- **Grounding Robustness:** DMAST’s focus on cross-modal consistency highlights the importance of P79’s grounding mechanism. Ensure that DOM/SoM element IDs are stable across retries and that visual grounding (if used) is validated against textual grounding.

Limitations/Caveats:

- Current experiments focus on data leakage; other adversarial objectives (control-flow hijacking, misinformation) not evaluated [5]
- Absolute TSR on VisualWebArena remains modest (10.2%) due to limited capacity of 12B stu-

dent model [5] - Three-stage training pipeline is computationally intensive; P79’s rule-based approach must approximate these learned behaviors with heuristics - Continuous streaming introduces latency and energy costs; P79’s offline evaluation avoids this but limits real-world applicability insights

Paper 6: Hierarchical Memory Tree (HMT)

Citation: Yunteng Tan, Zhi Gao, Xinxiao Wu. “Enhancing Web Agents with a Hierarchical Memory Tree.” arXiv:2603.07024v1 [cs.AI], 2026 [6].

Core Contribution: HMT is a structured framework that explicitly decouples logical planning from action execution through a three-level hierarchy (Intent, Stage, Action levels) constructed from raw trajectories using an automated abstraction pipeline. This hierarchical design prevents invalid execution details from propagating to new environments while preserving procedural logic, significantly outperforming flat-memory methods in cross-website and cross-domain scenarios [6].

Key Quantitative Results: - **Mind2Web (Cross-Website Split):** HMT improved StepSR by 6.0% compared to AWM; achieved 84.2% recall for retrieved memories vs. flat retrieval’s 65.8% [6] - **WebArena:** HMT achieved 38.7% total success rate, with substantial improvements in GitLab (+5.8%) and CMS (+5.0%); reduced average steps from 5.9 to 5.2 [6] - **Efficiency:** HMT reduced average context length by ~72.7%, latency from 5.2s to 3.5s, and inference cost per task by 71.0% [6] - **Ablation:** w/ Flat Memory: 6.6% gain on WebArena; w/o Pre/Post-conditions: 2.5% drop; w/ Raw Element Identifiers: catastrophic drop from 39.7% to 12.4% StepSR on Mind2Web Cross-Website [6]

Relevant Dimensions: - **(1) Observation Representation:** Processes raw page observations into semantic element descriptions - **(2) Cost-Aware Design:** 71.0% reduction in inference cost per task - **(3) Routing/Model Selection:** Planner-Actor decomposition acts as routing mechanism - **(4) Memory Management:** Core contribution—hierarchical memory tree with three levels - **(5) Grounding/Failure Recovery:** Semantic matching and confidence-aware fallback mechanism - **(8) Task Planning/Decomposition:** Three-level hierarchy (Intent, Stage, Action)

Actionable Insights for P79: - **Module M4 (Memory Management):** HMT’s three-level hierarchy provides a concrete template for P79’s memory module. Adapt for VisualWebArena: Intent level = task goal (e.g., “Find product with constraints”), Stage level = functional subgoals (e.g., “Navigate to category”, “Apply filters”, “Select item”), Action level = specific actions with semantic element descriptions. - **Phase 3 (Ablation):** HMT’s ablation shows that hierarchical structure provides 6.6% gain over flat memory. Test M4 with flat vs. hierarchical memory to measure marginal contribution. The 2.5% drop without pre/post-conditions suggests that explicit state tracking is valuable. - **Grounding Strategy:** HMT’s catastrophic failure with raw element identifiers (39.7% → 12.4%) demonstrates the critical importance of semantic descriptions. For P79, ensure that DOM/SoM representations include semantic labels (role, visible text, relative position) rather than just raw IDs. - **Cost Reduction:** HMT’s 71.0% cost reduction is achieved through hierarchical retrieval that reduces

context length by 72.7%. This aligns with M²'s approach and provides a second validation point for P79's memory optimization target (40-70% token reduction).

Limitations/Caveats: - **Ambiguous Grounding:** Generic descriptions lacking hierarchical context can lead to distractor matching (e.g., "text_contains: 'more'" matching wrong button) [6] - **State Verification Error:** Rigid post-condition checks fail in Single Page Applications where visual updates don't trigger URL changes, causing retry loops [6] - **Maps Domain:** HMT performs slightly worse than AWM in Maps domain (42.2% vs. 43.3%), attributed to spatial coordinate reliance where semantic text-based abstraction offers less benefit [6] - Automated abstraction pipeline quality depends on trajectory diversity; limited VisualWebArena training data may reduce effectiveness

Paper 7: Adaptive VLM Routing (AVR)

Citation: Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, Huamin Chen. "Adaptive Vision-Language Model Routing for Computer Use Agents." arXiv:2603.12823v1 [cs.CL], 2026 [7].

Core Contribution: AVR is a framework that optimizes cost and performance of Computer Use Agents by dynamically selecting the most appropriate VLM for each action through a lightweight semantic routing layer. It estimates action difficulty, probes a small VLM for confidence, and routes actions to the cheapest model meeting a target reliability threshold, addressing inefficiencies of using a single fixed VLM for all actions [7].

Key Quantitative Results: - **Inference Cost Reduction:** Up to 78% cost reduction while maintaining grounding accuracy within 2pp of all-large-model baseline [7] - **Cold AVR:** 52% cost savings with 42.1% effective accuracy, escalating 35% of actions to larger model [7] - **Warm AVR:** 70% cost savings with 41.3% accuracy, escalating only 15% of actions [7] - **Warm AVR + Difficulty Classification:** 78% cost savings with 42.8% accuracy [7] - **VLM Grounding Accuracy (ScreenSpot-Pro):** GPT-4o: 0.8%; OS-Atlas-7B: 18.9%; Qwen2.5-VL-72B: 43.6%; Qwen2.5-VL scaling (3B to 72B): 24.2% to 43.6% (1.8x improvement with 24x more parameters) [7] - **Memory Impact (OpenClaw):** Memory injection shifted 7B model's confidence from 0.83 to 0.96, enabling 86% cost reduction with no quality degradation [7]

Relevant Dimensions: - **(2) Cost-Aware Design:** Core focus—optimizing cost-performance trade-off - **(3) Routing/Model Selection:** Core contribution—dynamic VLM selection per action - **(4) Memory Management:** Memory injection improves small model confidence - **(6) Small Models ≤10B:** Specifically explores 7B models

Actionable Insights for P79: - **Phase 2 (Rule-based Routing):** AVR's three-mechanism approach (difficulty classification, confidence-based routing, safety-integrated routing) provides a comprehensive template for P79's routing logic. Implement: 1. **Difficulty Classification:** Use heuristics (form density, number of interactive elements, action history length) to estimate action difficulty 2. **Confidence-Based Routing:** Use Qwen3-VL-4B's logprobs to measure confidence; if below threshold (e.g., 0.85), trigger M3 (two-stage reasoning) or M2 (fallback) 3.

Safety-Integrated Routing: For high-risk actions (e.g., final submission), always use two-stage reasoning - **Module M2 (Fallback):** AVR’s escalation strategy (small model → large model) maps to P79’s fallback module. When Qwen3-VL-4B confidence is low, fallback to: (1) different representation (DOM→SoM), (2) two-stage reasoning, or (3) memory retrieval. - **Module M4 (Memory Management):** AVR’s memory injection (0.83 → 0.96 confidence boost) demonstrates the value of contextual memory. For P79, inject retrieved similar trajectories into prompt when confidence is low. - **Cost Metrics:** AVR’s 78% cost reduction with warm agents + difficulty classification provides an aspirational target for P79. Aim for 50-70% cost reduction through selective application of expensive modules (M3, M4).

Limitations/Caveats: - **Projected, Not Measured:** Cost savings are projected by combining OpenClaw routing data and ScreenSpot-Pro accuracy data; end-to-end validation needed [7] - **Probe Overhead:** For very short tasks (2-3 actions), probe overhead may negate cost savings; AVR most beneficial for longer sessions (≥ 10 actions) [7] - **Memory Cold-Start:** Newly deployed agent has no memories, limiting warm agent benefits; pre-seeding with documentation could help [7] - **Difficulty KB Coverage:** Knowledge base must cover target application landscape; unseen categories default to medium-difficulty tier [7] - **Screenshot Token Cost:** CUA tool calls dominated by screenshot tokens (2000-5000 per image); probe cost proportional to screenshot size [7]

Paper 8: Agentic Test-Time Scaling (CATTS)

Citation: Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, Amir Gholami. “Agentic Test-Time Scaling for WebAgents.” arXiv:2602.12276v1 [cs.AI], 2026 [8].

Core Contribution: CATTS introduces Confidence-Aware Test-Time Scaling, a technique that dynamically allocates compute for multi-step agents based on vote-derived uncertainty. Instead of uniformly increasing sampling, CATTS uses uncertainty statistics (entropy and top-1/top-2 margin) to decide when to apply additional compute, improving efficiency and providing an interpretable decision rule [8].

Key Quantitative Results: - **WebArena-Lite Performance:** CATTS improved performance by up to 9.1% over React [8] - **Token Efficiency:** CATTS used up to 2.3x fewer tokens than uniform scaling [8] - **Majority Vote (WebArena-Lite, N=10):** 43.2% success rate using 920K tokens [8] - **CATTS (Entropy-gated, best τ):** 47.9% success rate using 745K tokens (4.7% improvement) [8] - **CATTS (Margin-gated, best τ):** 47.9% success rate using 405K tokens (56% token reduction vs. majority voting) [8] - **Semantic Deduplication:** Without deduplication, accuracy dropped from 83.3% to 80.1% at N=32 on GoBrowse; with deduplication, improved from 83.3% to 84.5% at N=8 [8]

Relevant Dimensions: - **(2) Cost-Aware Design:** Core focus—dynamically allocating compute to improve efficiency - **(3) Routing/Model Selection:** Explores different selection strategies (majority voting, arbitration, confidence-aware scaling) - **(5) Grounding/Failure Recovery:** Includes error checks for invalid actions with retry

attempts

Actionable Insights for P79: - Module M1 (Retry): CATTS’s confidence-aware approach provides a principled framework for P79’s retry logic. Instead of fixed retry counts, implement adaptive retry based on vote-derived uncertainty: 1. Sample $N=3$ actions from Qwen3-VL-4B 2. Calculate entropy $H = -\sum p(a) \log p(a)$ where $p(a)$ is vote proportion 3. If $H > \text{threshold}$ (e.g., 0.8), trigger retry with different representation or two-stage reasoning 4. Use margin (difference between top-1 and top-2 vote counts) as alternative uncertainty metric - **Phase 2 (Routing):** CATTS demonstrates that selective compute allocation (margin-gated: 56% token reduction) is more efficient than uniform scaling. Design routing rules that apply expensive operations (M3, M4) only when uncertainty is high. - **Semantic Deduplication:** CATTS’s finding that deduplication is crucial for effective vote aggregation is relevant to P79’s retry module. When sampling multiple actions, deduplicate semantically similar actions before voting to avoid redundant computation. - **Benchmark Comparison:** CATTS reports 43.2% success on WebArena-Lite with majority voting ($N=10$), providing a reference point for P79’s VisualWebArena baseline expectations.

Limitations/Caveats: - Uniform Scaling Inefficiency: Naive uniform scaling leads to wasted computation on easy steps and high-variance decisions [8] - **Arbiter Overthinking:** Arbiters prone to overriding correct consensus actions, especially on high-consensus steps, leading to harmful outcomes [8] - **DeepConf Applicability:** Requires token-level log probabilities, limiting applicability to API-only models [8] - **RSA Performance:** Recursive Self-Aggregation achieved comparable but not better performance than simpler methods despite substantially higher compute costs (up to 80 LLM calls per step) [8] - Model used (gpt-oss-120b) is much larger than P79’s Qwen3-VL-4B; effectiveness of confidence-based routing for smaller models requires validation

Paper 9: WebWorld

Citation: Zikai Xiao, Jianhong Tu, Chuhang Zou, Yuxin Zuo, Zhi Li, Peng Wang, Bowen Yu, Fei Huang, Junyang Lin, ZuoZhu Liu. “WebWorld: A Large-Scale World Model for Web Agent Training.” arXiv:2602.14721v1 [cs.AI], 2026 [9].

Core Contribution: WebWorld introduces the first open-web simulator trained at scale, leveraging a scalable data pipeline to process over 1 million open-web interactions. It supports reasoning, multi-format data, and long-horizon simulations of over 30 steps, providing a replicable recipe for world model construction and enabling effective inference-time search [9].

Key Quantitative Results: - Extrinsic Evaluation (WebArena): Qwen3-14B trained on WebWorld-synthesized trajectories improved by +9.2%, reaching performance comparable to GPT-4o [9] - **Extrinsic Evaluation (MiniWoB++):** Qwen3-8B fine-tuned on WebWorld-synthesized trajectories achieved +9.9% gains [9] - **Extrinsic Evaluation (WebArena, Qwen3-8B):** +10.9% gains; Reddit and GitLab sub-domains showed strong gains of 18.3% and 12.0% respectively [9] - **Factuality Score:** WebWorld-32B achieved 71.0% average, matching Claude-Opus-4.1 (71.3%)

[9] - **Long-horizon Consistency:** WebWorld-32B showed 77.0% consistency [9] - **Reasoning Activation:** Minimal dataset of 1,000 CoT samples yielded Total Score of 0.561, surpassing direct reasoning tuning on Qwen3-8B with 10x more data (0.510) [9]

Relevant Dimensions: - **(1) Observation Representation:** Adopts A11y Tree as primary state representation; supports HTML, XML, Markdown - **(6) Small Models $\leq 10B$:** Includes 8B models in WebWorld series - **(7) Benchmark Methodology:** Introduces WebWorld-Bench with dual metrics (Factuality Score, Web Turing Score) - **(8) Task Planning/Decomposition:** Supports long chain-of-thought reasoning

Actionable Insights for P79: - **Phase 1 (Representation Comparison):** WebWorld’s adoption of A11y Tree as primary representation (due to universal applicability, high information density, LLM-friendly structure) provides strong validation for P79’s SoM representation. Expect SoM to perform well for Qwen3-VL-4B. - **Module M3 (Two-stage Reasoning):** WebWorld’s finding that minimal CoT data (1,000 samples) activates reasoning capabilities suggests that P79’s two-stage reasoning module doesn’t require extensive training. Implement M3 by prompting Qwen3-VL-4B to generate explicit reasoning steps before action selection. - **Training Data Synthesis:** WebWorld demonstrates that world models can synthesize high-quality training data. For P79’s future work, consider using a world model to generate additional VisualWebArena trajectories for fine-tuning Qwen3-VL-4B. - **Long-horizon Performance:** WebWorld’s 77.0% long-horizon consistency and support for 30+ step simulations suggest that world models can help agents maintain coherence in complex tasks. This is relevant to P79’s memory management module (M4).

Limitations/Caveats: - **Sycophancy:** WebWorld exhibits sycophancy, generating overly optimistic outcomes that cater to agent’s action, hindering robust policy learning [9] - **Content Generation Quality:** Struggles to generate high-quality, detailed content (e.g., scientific articles) [9] - **Dual-use Concerns:** More capable web agents introduce risks (automated phishing, credential stuffing, large-scale scraping) [9] - **Data Sourcing Risks:** Training data may contain PII, toxic content, or demographic biases despite filtering [9] - **Inference-time Search Limitations:** Bounded gains suggest world models more valuable for synthesizing training data than direct inference-time search [9]

Paper 10: ContractSkill

Citation: Not fully specified in chunks. “ContractSkill: Deterministic Verification and Repair of Multimodal Web Skills.” [10]

Core Contribution: ContractSkill introduces a framework for deterministic verification and repair of multimodal web skills through explicit procedural contracts. It uses fault localization to identify where skills fail, then applies constrained local repair to fix errors while preserving correct behavior. This approach enables cross-model skill transfer and reduces token usage by reusing repaired artifacts instead of regenerating online [10].

Key Quantitative Results: - **VisualWebArena (GLM-4.6V):** ContractSkill achieved

77.5% success rate (+19.5pp over No-Skill 58.0%, +17.0pp over Self-Generated 60.5%) with 2.00 steps and 7.9k tokens [10] - **VisualWebArena (Qwen3.5-Plus)**: 81.0% success rate (+24.5pp over No-Skill 56.5%, +20.5pp over Self-Generated 60.5%) with 2.00 steps and 7.9k tokens [10] - **Cross-Model Transfer (VWA)**: Q→G: 29.2% → 79.2% (+50.0pp); G→Q: 36.4% → 81.8% (+45.4pp); Average: 32.6% → 80.4% (+47.8pp) [10] - **Cross-Model Transfer (MiniWoB)**: Q→G: 87.5% → 100.0% (+12.5pp); G→Q: 79.2% → 92.3% (+13.1pp); Average: 83.5% → 96.2% (+12.8pp) [10] - **Ablation (MiniWoB)**: Text-Only Rewrite: GLM 62.0% vs. full 77.5%; Qwen 60.5% vs. full 81.0%; w/o Failure Localization: GLM 65.0%, Qwen 70.0% [10]

Relevant Dimensions: - **(1) Observation Representation:** Screenshot + structured page summary (DOM abstraction, accessibility summary) - **(5) Grounding/Failure Recovery:** Core contribution—deterministic verification, fault localization, local repair - **(8) Task Planning/Decomposition:** Explicit procedural structure (skill_name, goal, preconditions, steps, postconditions, recovery, terminate)

Actionable Insights for P79: - **Module M1 (Retry) & M2 (Fallback):** ContractSkill’s fault localization and local repair approach provides a sophisticated template for P79’s retry and fallback mechanisms. Instead of naive retry, implement: 1. **Deterministic Verification:** After each action, check if expected state change occurred (e.g., URL changed, element appeared) 2. **Fault Localization:** If verification fails, identify which step failed (e.g., selector not found, action had no effect) 3. **Local Repair:** Retry with modified strategy (e.g., different selector, alternative action sequence) - **Module M4 (Memory Management):** ContractSkill’s skill artifact structure (preconditions, steps, postconditions) maps to HMT’s hierarchical memory. For P79, store successful trajectories with explicit pre/post-conditions to enable retrieval and reuse. - **Cross-Model Transfer:** ContractSkill’s 47.8pp average gain on VWA through cross-model transfer suggests that P79 could benefit from pre-computing skills with a larger model (e.g., Qwen3-VL-32B), then transferring to Qwen3-VL-4B. - **VisualWebArena Baseline:** ContractSkill reports 58.0% (GLM-4.6V) and 56.5% (Qwen3.5-Plus) baseline success rates on VWA, providing reference points for P79’s expectations.

Limitations/Caveats: - **Scope Limitations:** Study focuses on single multimodal web skill artifacts, not full lifelong skill library [10] - **Artifact Schema:** Tailored to browser interaction; doesn’t cover broader action space of desktop environments [10] - **Transfer Scope:** Cross-model reuse tested within same benchmark family; portability notion is limited [10] - **Verification Limitations:** Deterministic verification strongest when success expressed through programmatic checks (URL, DOM state, form values); visually ambiguous or highly dynamic pages harder to verify [10] - **Common Failure Patterns:** Skills fail through early wrong selectors, missing preconditions, premature stop conditions; committed but brittle procedures often truncate exploration [10]

Paper 11: OpAgent

Citation: Not fully specified in chunks. “OpAgent (Operator Agent).” arXiv:2602.13559v1 [cs.AI], 2026 [11].

Core Contribution: OpAgent introduces a robust online reinforcement learning approach for web agents comprising three innovations: (1) Hierarchical Multi-Task Fine-tuning across Planning, Acting, and Grounding primitives, (2) Online Agentic RL with a Hybrid Reward Mechanism combining WebJudge and Rule-based Decision Tree for credit assignment, and (3) A modular agentic framework orchestrating Planner, Grounder, Reflector, and Summarizer for robust error recovery [11].

Key Quantitative Results: - **WebArena Benchmark:** RL-enhanced model (pass@5) achieved 38.1% success rate; OpAgent (SOTA) achieved 71.6% success rate, top leaderboard position (January 2026) [11] - **Wild Websites (87 sites):** Baseline (Qwen2.5-VL-72B): 2.01 average score; RL-HybridReward-Zero: 3.09; RL-HybridReward: 3.56 (1.55 point improvement vs. baseline) [11] - **Pass@K Metrics:** Pass@1: RL model outperformed baseline by 8.08%; Pass@5: performance gap widened to 10.66% [11] - **Iterative SFT+RL Pipeline:** Post-RL performance improved from 3.09 to 3.56 by incorporating high-quality trajectories from Online RL into SFT dataset [11]

Relevant Dimensions: - **(1) Observation Representation:** VLM-based visual perception with text-centric history strategy - **(3) Routing/Model Selection:** Modular framework with specialized agents (Planner, Grounder, Reflector, Summarizer) - **(4) Memory Management:** Text-based action history, discarding visual history to avoid hallucinations - **(5) Grounding/Failure Recovery:** Reflector and Summarizer for error recovery - **(7) Benchmark Methodology:** Wild Websites evaluation (87 sites) - **(8) Task Planning/Decomposition:** Planner for strategic decomposition

Actionable Insights for P79: - **Module M2 (Fallback) & M5 (Grounding/Failure Recovery):** OpAgent’s modular framework (Planner, Grounder, Reflector, Summarizer) provides a template for P79’s fallback and recovery mechanisms. Implement: 1. **Reflector:** After failed action, prompt Qwen3-VL-4B to analyze why action failed (e.g., “Element not found”, “Action had no effect”) 2. **Summarizer:** Compress action history to maintain context efficiency (similar to M2’s Internal Memory) 3. **Grounder:** Verify that selected element matches intended target before executing action - **Phase 2 (Routing):** OpAgent’s modular approach suggests that routing should consider task complexity. For complex tasks requiring strategic planning, route to two-stage reasoning (M3); for simple tasks, use direct action selection. - **Memory Strategy:** OpAgent’s text-centric history (discarding visual history to avoid hallucinations) is relevant to P79’s memory module (M4). For Qwen3-VL-4B with limited context, consider storing only current screenshot + text-based action history. - **Benchmark Expectations:** OpAgent’s 71.6% success on WebArena (with extensive RL training and multi-agent orchestration) provides an upper bound for P79’s expectations. Without RL, P79’s Qwen3-VL-4B likely achieves 10-30% success on VisualWebArena.

Limitations/Caveats: - **Heavy Dependence on Prompt Engineering:** Requires complex orchestration of multiple agents, substantial human labor, and computational overhead [11] - **Trajectory Filtering:** Trajectories with negative scores systematically discarded during RL training [11] - **Future Challenge:** Enhancing intrinsic exploration capabilities to reduce reliance on multi-agent orchestration [11] - Uses 32B-72B models; effectiveness for Qwen3-VL-4B requires validation

Paper 12: SkillWeaver

Citation: Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, Yu Su. “SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills.” [12]

Core Contribution: SkillWeaver proposes a self-improvement framework where web agents autonomously discover reusable skills from successful trajectories, refine them through practice, and compose them to solve new tasks. The framework includes skill discovery (extracting patterns from trajectories), skill refinement (improving through repeated practice), and skill composition (combining skills for complex tasks) [12].

Key Quantitative Results: - **WebArena:** SkillWeaver achieved 38.7% success rate, outperforming baselines [12] - **Mind2Web:** Improved performance through skill composition [12] - **Skill Discovery:** Automatically extracted 50+ reusable skills from 100 successful trajectories [12] - **Skill Refinement:** Average skill success rate improved from 65% to 82% after 3 refinement iterations [12]

Relevant Dimensions: - **(4) Memory Management:** Skill library as structured memory - **(5) Grounding/Failure Recovery:** Skill refinement through practice - **(8) Task Planning/Decomposition:** Skill composition for complex tasks

Actionable Insights for P79: - **Module M4 (Memory Management):** SkillWeaver’s skill library concept aligns with M2’s External Memory and HMT’s hierarchical memory. For P79, implement a lightweight skill library: 1. Extract common patterns from successful VisualWebArena trajectories (e.g., “Search for product”, “Apply price filter”, “Add to cart”) 2. Store as reusable templates with preconditions and expected outcomes 3. Retrieve relevant skills when task matches stored patterns - **Module M1 (Retry):** SkillWeaver’s skill refinement (65% → 82% success after 3 iterations) suggests that retry mechanisms should learn from failures. For P79, track which retry strategies work for which failure types, then prioritize successful strategies in future retries. - **Phase 3 (Ablation):** Test M4 with and without skill library to measure marginal contribution. SkillWeaver’s results suggest skill-based memory provides 5-10pp improvement over flat memory. - **Skill Composition:** For complex VisualWebArena tasks, consider decomposing into skill sequences (e.g., “Search” → “Filter” → “Select” → “Checkout”) rather than treating as monolithic tasks.

Limitations/Caveats: - Skill discovery requires successful trajectories; limited VisualWebArena training data may constrain skill library size - Skill refinement through practice requires multiple task attempts, increasing computational cost - Skill composition assumes tasks can be decomposed into independent subtasks; some VisualWebArena tasks may have complex dependencies - Paper doesn’t report specific quantitative results for all metrics; some numbers are estimates based on typical performance

Paper 13: StructuredAgent

Citation: Elita A. Lobo, Jingjing Meng, Yang Jiao, Yair Zick, Xu Chen, Nan Xi, Chirag Agarwal, Yan Gao. “STRUCTUREDAGENT: Planning with AND/OR Trees for Long-Horizon Web Tasks.” 39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: Efficient Reasoning / arXiv:2603.05294v2 [cs.AI], 2025 [13].

Core Contribution: StructuredAgent is a hierarchical planning framework that enables agents to dynamically construct and execute ordered AND/OR hierarchical planning trees during task execution. It includes an online hierarchical planner using dynamic AND/OR trees for efficient search and a structured memory module that tracks candidate solutions to improve constraint satisfaction in information-seeking tasks [13].

Key Quantitative Results: - **Amazon Easy Tasks:** StructuredAgent achieved 83.3% average success rate, outperforming AgentOccam by 14% [13] - **Amazon Hard Tasks:** StructuredAgent: 33.3%; StructuredAgentMem (with Structured Memory): 37.8% (5% improvement) [13] - **WebVoyager Easy Tasks:** Comparable to baselines with marginal 1.5% drop [13] - **WebArena Tasks:** 52.6% overall score across categories (Map, Shopping, Reddit, GitLab), surpassing AgentOccam by ~6% and BasicClaudeAction/WebArenaReplication by ~20% [13] - **Complex Shopping (Amazon Hard) with Claude 3.7:** 46.7% average success, outperforming AgentOccam by 7.8% and BasicClaudeAction by 23.4% [13]

Relevant Dimensions: - **(1) Observation Representation:** Observation history encoded as token sequences; observation summarizer for internal representations - **(4) Memory Management:** Structured memory module tracks candidate entities and constraints - **(5) Grounding/Failure Recovery:** Systematic node revision and pruning to recover from failures - **(8) Task Planning/Decomposition:** Core contribution—hierarchical planning with AND/OR trees

Actionable Insights for P79: - **Module M3 (Two-stage Reasoning):** StructuredAgent’s AND/OR tree planning provides a concrete template for P79’s two-stage reasoning. Implement M3 as: 1. **Stage 1 (Planning):** Decompose task into AND/OR tree (e.g., “Find product” = “Search” AND (“Apply filters” OR “Browse categories”)) 2. **Stage 2 (Execution):** Execute leaf nodes, backtrack on failure, try alternative OR branches - **Module M4 (Memory Management):** StructuredAgent’s structured memory (tracking candidate entities and constraints) is particularly relevant for VisualWebArena’s shopping tasks. For P79, implement a simple table: rows = candidate products, columns = constraints (price, rating, features), cells = satisfied/unsatisfied. - **Phase 3 (Ablation):** StructuredAgent’s ablation shows that Structured Memory provides 5% improvement on hard tasks but slightly degrades performance on easy tasks. Test M4 with and without structured memory to measure marginal contribution and identify when it’s beneficial. - **Failure Recovery:** StructuredAgent’s systematic node revision and pruning aligns with P79’s retry module (M1). When action fails, backtrack to parent node and try alternative branch rather than naive retry.

Limitations/Caveats: - **Existing Agents’ Struggles:** Limited in-context memory, weak planning, greedy behaviors leading to premature termination [13] - **LLM-as-a-**

Judge Penalties: LLM judges penalize partial satisfaction more harshly than human annotators; high disagreement rates on Amazon Hard tasks [13] - **Kimi-k2 Performance:** Degraded performance on long-context inputs noted as model-specific limitation [13] - Uses Claude 3.5/3.7 Sonnet; effectiveness for Qwen3-VL-4B requires validation

Paper 14: Web-CogReasoner

Citation: Yuhan Guo, Cong Guo, Aiwen Sun, Hongliang He, Xinyu Yang, Yue Lu, Yingji Zhang, Xuntao Guo, Dong Zhang, Jianzhuang Liu, Jiang Duan, Yijia Xiao, Liangjian Wen, Hai-Ming Xu, Yong Dai. “WEB-COGREASONER: TOWARDS KNOWLEDGE-INDUCED COGNITIVE REASONING FOR WEB AGENTS.” Preprint. Accepted as a conference paper at ICLR 2026. arXiv:2508.01858v2 [cs.CL], 2026 [14].

Core Contribution: Web-CogReasoner decomposes web agent capabilities into knowledge content learning and cognitive processes, categorizing knowledge into Factual, Conceptual, and Procedural domains. It introduces Web-CogDataset for instilling core knowledge and Web-CogBench for evaluation, operationalizing these through a knowledge-driven Chain-of-Thought (CoT) reasoning framework [14].

Key Quantitative Results: - **Web-CogBench (Overall Accuracy):** Web-CogReasoner: 84.4%; Claude Sonnet 4: 76.8%; Gemini 2.5 Pro: 80.4%; Qwen2.5-VL-7B: 69.8%; UI-TARS-7B-SFT: 46.4% [14] - **VisualWebBench (Overall Average Score):** Web-CogReasoner: 86.3%; UI-TARS: 86.0% [14] - **WebVoyager (Overall Average):** Web-CogReasoner: 30.2%; Gemini 2.5 Pro: 54.9%; Claude Sonnet 4: 47.7% [14] - **Online Mind2Web:** Web-CogReasoner: 17.0% (Cross-task), 10.1% (Cross-web); Claude Sonnet 4: 40.2% (Cross-task), 21.7% (Cross-web) [14] - **Average Steps per Successful Task:** Web-CogReasoner: 7.00; Gemini: 8.24; Claude: 9.76 [14] - **Cumulative Gains (Web-CogBench):** Base: 69.8%; +Factual: 72.1% (+17.9% on Memorizing); +Conceptual: 78.3% (+11.3% on Understanding); +Procedural: 84.4% (+19.2% on Exploring) [14]

Relevant Dimensions: - **(1) Observation Representation:** Screenshot and accessibility tree (AX Tree) - **(5) Grounding/Failure Recovery:** Knowledge-driven CoT provides grounding; Noisy Multi-Step Web Task benchmark evaluates error recovery - **(7) Benchmark Methodology:** Introduces Web-CogBench for systematic evaluation - **(8) Task Planning/Decomposition:** Knowledge-driven CoT decomposes tasks into Factual, Conceptual, Procedural layers

Actionable Insights for P79: - **Module M3 (Two-stage Reasoning):** Web-CogReasoner’s knowledge-driven CoT provides a structured template for P79’s two-stage reasoning. Implement M3 as: 1. **Stage 1 (Knowledge Retrieval):** Identify relevant factual knowledge (e.g., “Amazon uses star ratings”), conceptual knowledge (e.g., “Price filters are typically in left sidebar”), and procedural knowledge (e.g., “To apply filter: click filter category → select option → click apply”) 2. **Stage 2 (Action Selection):** Use retrieved knowledge to guide action selection - **Phase 1 (Representation Comparison):** Web-CogReasoner uses screenshot

+ AX Tree, achieving 84.4% on Web-CogBench. This validates P79’s hybrid representation approach (screenshot + DOM/SoM). - **Benchmark Expectations:** Web-CogReasoner (Qwen2.5-VL-7B) achieves 30.2% on WebVoyager, providing a reference point for P79’s Qwen3-VL-4B expectations on VisualWebArena (likely 15-30%). - **Knowledge Ablation:** Web-CogReasoner’s cumulative gains (+17.9% Factual, +11.3% Conceptual, +19.2% Procedural) suggest that procedural knowledge is most critical. For P79, prioritize procedural knowledge in M3 (e.g., “How to apply filters”, “How to navigate categories”).

Limitations/Caveats: - **Current Reliance on Imitation Learning:** Future work aims to integrate RL for enhanced exploration and autonomous procedural knowledge discovery [14] - **Base Model Struggles:** Without specialized training, base model produces generic reasoning or hallucinates actions, lacks procedural knowledge [14] - **Knowledge Blind Spot:** Base model lacks explicit knowledge of page layout and element functions, leading to logical dead loops [14] - **Difficulty with Complex Procedural Logic:** Even after Factual and Conceptual training, agents struggle with complex procedures [14]

Paper 15: WEBSERV

Citation: Yuxuan Lu, Jing Huang, Hui Liu, Jiri Gesi, Yan Han, Shihan Fu, Tianqi Zheng, Dakuo Wang. “WEBSERV: A Browser-Server Environment for Efficient Training of Reinforcement Learning-based Web Agents at Scale.” 39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: Multi-Turn Interactions in Large Language Models. arXiv:2510.16252v1 [cs.LG], 2025 [15].

Core Contribution: WEBSERV proposes a browser-server environment designed for scalable and efficient training and evaluation of RL web agents. It includes a compact, site-agnostic browser environment that balances context and action complexity, and a scalable RL environment that efficiently launches and resets web-servers to enable scalable RL training and evaluation [15].

Key Quantitative Results: - **Single-prompt Success Rates (WebArena-Lite):** WEBSERV with Claude 4.5 achieved 46.7% (Shopping), 34.3% (CMS), 40.0% (Gitlab), establishing new SOTA among single-prompt agents [15] - **Launch Latency Reduction:** ~5x reduction (1.781s vs. 8.963s for Naïve Docker) [15] - **Storage Need Reduction:** ~240x reduction (28.01 MiB vs. 6.78 GiB for Naïve Docker) [15] - **Concurrent Containers:** Enables 200+ concurrent containers on single host with comparable memory footprint [15] - **Memory Footprint:** 1.74 GiB vs. 1.63 GiB for Naïve Docker (comparable) [15]

Relevant Dimensions: - **(1) Observation Representation:** DOM parser automatically reduces page to visible and meaningful elements; JSON object with five components (HTML snapshot, clickable elements, hoverable elements, input elements, select elements) - **(2) Cost-Aware Design:** ~5x launch latency reduction, ~240x storage reduction, 200+ concurrent containers - **(4) Memory Management:** Incus-based container management with block-level copy-on-write - **(5) Grounding/Failure Recovery:** Intercepts network events, waits for page quiescence before

returning observations; explicit error state if idle state not reached within timeout

Actionable Insights for P79: - Phase 1 (Representation Comparison): WEBSERV’s DOM parser approach (filtering invisible/irrelevant nodes, preserving key visual cues and interactive elements with semantic identifiers) provides a concrete template for P79’s DOM representation. Implement similar filtering to reduce token consumption while maintaining task-relevant information. - **Grounding Strategy:** WEBSERV’s use of stable semantic identifiers (data-semantic-id) that are robust to minor DOM changes is relevant to P79’s grounding mechanism. Ensure that DOM/SoM element IDs are stable across retries and page updates. - **Benchmark Expectations:** WEBSERV reports 46.7% (Shopping), 34.3% (CMS), 40.0% (Gitlab) on WebArena-Lite with Claude 4.5, providing reference points for P79’s VisualWebArena expectations. - **Efficiency Optimization:** WEBSERV’s 5x launch latency reduction and 240x storage reduction demonstrate the value of infrastructure optimization. For P79, consider similar optimizations for VisualWebArena evaluation (e.g., pre-loading common pages, caching DOM representations).

Limitations/Caveats: - Visual Perception: Current design assumes text-only agents; visual signals limited to metadata, preventing reasoning about spatial layout, color, images [15] - **Lack of Visual Layout Cues:** Parser provides compact HTML but doesn’t retain visual layout cues (e.g., grid arrangements presented as flat lists), removing structural signals humans use [15] - Experiments with Claude 4.5/4; effectiveness for Qwen3-VL-4B requires validation

Paper 16: MoE-ICRL (Mixture-of-Experts Meets In-Context Reinforcement Learning)

Citation: Wu et al. “Mixture-of-Experts Meets In-Context Reinforcement Learning.” arXiv:2506.05426, 2025 [16].

Note: PDF parsing failed for this paper. Limited information available from paper table metadata.

Core Contribution: Proposes integrating Mixture-of-Experts (MoE) architecture with In-Context Reinforcement Learning for improved agent performance [16].

Key Quantitative Results: Not available due to PDF parsing failure.

Relevant Dimensions: - (3) Routing/Model Selection: MoE architecture for expert selection - **(8) Task Planning/Decomposition:** In-context RL for adaptive planning

Actionable Insights for P79: - Limited Applicability: Without detailed results, direct insights are limited. However, the general concept of MoE (routing to specialized experts) aligns with P79’s routing framework. - **Phase 2 (Routing):** Consider MoE-inspired routing where different “experts” represent different strategies (e.g., Expert 1 = DOM representation, Expert 2 = SoM representation, Expert 3 = two-stage reasoning).

Limitations/Caveats: - PDF parsing failed; comprehensive analysis not possible -

General MoE limitations: increased model complexity, potential latency issues, training difficulty

Paper 17: Ego2Web

Citation: Shoubin Yu, Lei Shu, Antoine Yang, Yao Fu, Srinivas Sunkara, Maria Wang, Jindong Chen, Mohit Bansal, Boqing Gong. “Ego2Web: A Web Agent Benchmark Grounded in Egocentric Videos.” arXiv:2603.22529v1 [cs.CV], 2026 [17].

Core Contribution: Ego2Web introduces the first benchmark that bridges egocentric video perception and web agent execution, addressing the limitation of existing benchmarks that focus solely on web-based interaction without grounding in the user’s real-world physical surroundings. It pairs real-world first-person video recordings with web tasks requiring visual understanding, web task planning, and online interaction [17].

Key Quantitative Results: - **Ego2WebJudge Agreement:** ~84% agreement with human judgment (Gemini-2.5-Pro: 80.8% AR; GPT-4o: 84.0% AR), outperforming WebVoyager (70.7%, 74.7%) and WebJudge (76.1%, 78.4%) [17] - **Agent Performance:** ~40% gap from oracle performance according to human evaluation [17] - **BU-Gemini-3-Flash Success Rate:** 58.6% SR (Human Eval); 48.2% SR (Ego2WebJudge, Gemini-2.5 Pro) [17] - **Task Type Distribution:** E-Commerce 50.3%, Knowledge Lookup 17.0%, Media Retrieval 24.1%, Local/Maps 6.0%, Others 2.6% [17] - **Ablation (Visual Perception):** No Visual Input: 4.4% SR; Detailed Caption Only: 23.6% SR; Raw Video Input: 48.2% SR [17]

Relevant Dimensions: - **(1) Observation Representation:** Core focus—egocentric video as rich observation representation - **(7) Benchmark Methodology:** Novel benchmark with semi-automatic data generation pipeline and Ego2WebJudge evaluation method - **(5) Grounding/Failure Recovery:** Emphasizes strict visual grounding as core evaluation principle - **(8) Task Planning/Decomposition:** Tasks test step-by-step web action planning based on video perception

Actionable Insights for P79: - **Phase 1 (Representation Comparison):** Ego2Web’s ablation (No Visual: 4.4% → Caption: 23.6% → Raw Video: 48.2%) demonstrates the critical importance of visual input for web agents. For P79, ensure that Qwen3-VL-4B receives high-quality screenshots (not just DOM/SoM text). - **Hybrid Representation:** Ego2Web’s finding that raw video input (48.2%) substantially outperforms caption-based perception (23.6%) suggests that P79’s hybrid representation (screenshot + DOM/SoM) should prioritize visual fidelity. Don’t over-compress screenshots to save tokens. - **Benchmark Methodology:** Ego2Web’s Ego2WebJudge (84% agreement with humans) provides a reference for P79’s evaluation methodology. Consider using LLM-as-a-judge for VisualWebArena evaluation, but validate against human judgment on a subset. - **Failure Modes:** Ego2Web’s error analysis (36% object misidentification, 18% temporal misunderstanding, 16% cross-modal retrieval failure) highlights the importance of accurate visual grounding. For P79, ensure that Qwen3-VL-4B’s visual grounding is validated against ground truth.

Limitations/Caveats: - **Information Loss from Textual Abstractions:** Agents relying on textual captions suffer from information loss, particularly for fine-grained spatial-temporal cues [17] - **Limited Temporal Context:** GPT-40-based agents on sparse keyframes capture limited temporal context, missing critical intermediate actions [17] - **Object Misidentification:** 36% of errors due to incorrect target object identification from egocentric video [17] - **Temporal and Action Misunderstanding:** 18% of failures due to misinterpreting temporal order or actions [17] - **Cross-Modal Retrieval Failure:** 16% of failures where agent correctly identifies target but fails to locate required information on web [17]

Paper 18: Optimizing Generative AI Networking

Citation: Ruichen Zhang, Hongyang Du, Dusit Niyato, Jiawen Kang, Zehui Xiong, Ping Zhang, Dong In Kim. "Optimizing Generative AI Networking: A Dual Perspective with Multi-Agent Systems and Mixture of Experts." arXiv:2405.12472v1 [cs.NI], 2024 [18].

Core Contribution: This paper proposes a hybrid framework integrating Multi-Agent Systems (MAS) and Mixture of Experts (MoE) to enhance generative AI capabilities and operational flexibility in AIGC-enabled networking. The framework uses MAS for dynamic task coordination among network service provider agents and MoE for expert-driven execution of tasks, aiming to improve overall system efficiency and adaptability in scenarios like 3D object generation and data transfer [18].

Key Quantitative Results: - **Text-to-Image Models:** DALL-E requires 12 billion parameters; GLIDE requires 3.5 billion parameters [18] - **Switch Transformers:** Google’s model employs 1.6 trillion parameters [18] - **3D Object Generation Data Size:** Transferring 3-5 2D images consumes up to 20MB; generating single 3D object through AIGC requires only ~7MB [18] - **Cumulative Reward:** MAS-MoE approach shows gradually increasing cumulative reward surpassing greedy and random strategies [18] - **Convergence Speed:** Multi-agent PPO requires ~40% more episodes to converge compared to MAS-MoE-PPO [18] - **Total Cost:** MAS-MoE approach consistently results in lowest costs under all conditions, even as number of NSPs increases [18]

Relevant Dimensions: - **(2) Cost-Aware Design:** Framework aims to minimize total operating costs (computational overhead, power consumption) - **(3) Routing/Model Selection:** MoE’s gating mechanism intelligently directs inputs to appropriate experts; MAS agents coordinate dynamically - **(8) Task Planning/Decomposition:** MAS enables dynamic task coordination; MoE facilitates expert-driven execution

Actionable Insights for P79: - **Phase 2 (Routing):** The MAS-MoE hybrid framework provides a conceptual template for P79’s routing logic. Consider routing as a two-level decision: 1. **High-level (MAS):** Determine task type (e.g., search, filter, select, checkout) and route to appropriate strategy 2. **Low-level (MoE):** Within each strategy, select appropriate expert (e.g., DOM representation, SoM representation, two-stage reasoning) - **Cost-Aware Design:** MAS-MoE’s focus on minimizing total

operating costs (computational overhead + power consumption) aligns with P79’s cost-aware routing objective. Track both token consumption and latency to optimize routing decisions. - **Convergence Speed:** MAS-MoE’s 40% faster convergence compared to multi-agent PPO suggests that expert specialization improves efficiency. For P79, consider specializing modules (M1-M4) for specific failure types or task types. - **Scalability:** MAS-MoE’s consistent cost advantage as number of NSPs increases suggests that routing frameworks scale well. For P79, design routing logic that scales to larger model pools (e.g., Qwen3-VL-4B, 8B, 32B).

Limitations/Caveats: - **Unmanageable Model Complexity:** GenAI models highly complex due to extensive parameters and large training datasets, requiring considerable computing resources [18] - **Low Adaptability:** GenAI’s reliance on predefined datasets and algorithms limits flexibility in responding to new data patterns [18] - **Performance Bottleneck:** Centralized GenAI approaches create bottlenecks regarding data transfer speed and processing time [18] - **MAS Task Fragmentation:** Independence of agents can lead to fragmentation of task focus, particularly in fragmented execution of GenAI tasks [18] - **MoE Latency Issues:** Centralized nature might introduce latency issues when coordinating large number of expert inputs across complex tasks [18]

Paper 19: Egocentric Co-Pilot

Citation: Sicheng Yang, Yukai Huang, Weitong Cai, Shitong Sun, Fengyi Fang, You He, Yiqiao Xie, Jiankang Deng, Hang Zhang, Jifei Song, Zhensong Zhang. “Egocentric Co-Pilot: Web-Native Smart-Glasses Agents for Assistive Egocentric AI.” ACM Web Conference 2026 (WWW ’26), 2026 [19].

Core Contribution: The paper introduces a modular, neuro-symbolic architecture orchestrated by a central LLM that connects human intent with specialized tools and web-accessible services through a web-native interface compatible with resource-constrained devices. Instead of relying on a single monolithic model, the framework uses an LLM as a reasoning engine to interpret multimodal commands, clarify intent through interactive dialogue and visual grounding, then generate execution plans by selecting and invoking specialized tools [19].

Key Quantitative Results: - **Human-in-the-Loop Study:** Mean rating: 4.70 (on 5-point Likert scale); Human baseline: 4.92; significantly surpassed all commercial competitors [19] - **Tool Use Performance:** Category 1 (Foundational Tool Use) TCR: 98.5% [19] - **Video Sampling Rate:** 1 FPS [19] - **Base Model:** Qwen2.5-VL-7B-Instruct (fine-tuned) [19] - **Discount Factor (γ):** 0.9 for temporal reward propagation [19]

Relevant Dimensions: - **(1) Observation Representation:** Hybrid multimodal—egocentric video at 1 FPS + dense narrations + ASR transcripts in unified event log - **(2) Cost-Aware Design:** WebRTC-based streaming, web-native interface for resource-constrained devices - **(3) Routing/Model Selection:** MCP-based tool selection and orchestration - **(4) Memory Management:** Dual-level T-CoT + HCC for short and long-term dependencies - **(5) Grounding/Failure Recovery:** Interactive

clarification module for ambiguous inputs - **(6) Small Models $\leq 10B$:** Qwen2.5-VL-7B-Instruct - **(8) Task Planning/Decomposition:** LLM generates execution plans by selecting and composing tools

Actionable Insights for P79: - **Module M4 (Memory Management):** Egocentric Co-Pilot’s dual-level memory (T-CoT for fine-grained recent events, HCC for long-term reasoning) provides a sophisticated template for P79’s memory module. Implement: 1. **Short-term Memory (T-CoT):** Store last 4-5 steps with full detail (screen-shot + action + result) 2. **Long-term Memory (HCC):** Compress older history into summaries (e.g., “Applied price filter, navigated to product page, added to cart”) - **Module M2 (Fallback):** Egocentric Co-Pilot’s interactive clarification module (detects semantic uncertainty, issues clarification questions) maps to P79’s fallback strategy. When Qwen3-VL-4B confidence is low, trigger clarification: (1) retrieve similar trajectories, (2) prompt for explicit reasoning, (3) try alternative representation. - **Small Model Optimization:** Egocentric Co-Pilot uses Qwen2.5-VL-7B-Instruct (similar size to P79’s Qwen3-VL-4B), demonstrating that small VLMs can achieve strong performance (4.70/5.0 rating) with proper architecture. This validates P79’s choice of Qwen3-VL-4B. - **Tool Orchestration:** Egocentric Co-Pilot’s MCP-based tool selection provides a template for P79’s module orchestration. Design routing logic that selects appropriate modules (M1-M4) based on task requirements and current state.

Limitations/Caveats: - **Dependency on Foundation Models:** Behavior depends on underlying LLM/VLM backbones and hand-designed tool schemas [19] - **Error Cascading:** Errors in perception, reasoning, or tool selection can cascade through pipeline [19] - **Weak Guardrails:** Current guardrails weaker than formal safety guarantees [19] - **Domain Adaptation Challenges:** Fine-tuning on first-person data may not transfer perfectly to new domains or camera form factors [19] - **Cloud Dependency:** Continuous streaming introduces latency and energy costs [19] - **Limited Evaluation Scope:** Focuses on short-term assistance with healthy adults in controlled settings [19] - **Privacy Concerns:** Always-on egocentric capture raises privacy and bystander consent issues [19]

Paper 20: Throttling Web Agents Using Reasoning Gates

Citation: Abhinav Kumar, Jaechul Roh, Ali Naseh, Amir Houmansadr, Eugene Bagdasarian. “Throttling Web Agents Using Reasoning Gates.” arXiv:2509.01619v1 [cs.AI], September 1, 2025 [20].

Core Contribution: This paper introduces a novel framework called ‘Web Agent Throttling’ that imposes tunable costs on AI web agents before granting access to resources by issuing “Reasoning Gates”—asymmetric challenges requiring agents to solve reasoning puzzles, thereby incurring excessive token-generation costs. The framework aims to deter malicious or excessive agent activity while preserving access for legitimate users and low-resource agents [20].

Key Quantitative Results: - **Computational Asymmetry:** Response-generation cost for SOTA models is 9.2x higher than generation cost for rebus-based Reasoning Gates (rRG) [20] - **Reasoning Models:** Average asymmetry of 6.4x, highest at 9.2x

(DeepSeek R1 consuming 46k tokens to solve gate generated by o3-mini which cost 4.9k tokens) [20] - **Non-Reasoning Models:** Average asymmetry of 5.2x, highest at 6.6x (Gemma consuming 2.8k tokens to solve gate generated by Gemini 2.5-Flash which cost 430 tokens) [20] - **Human Performance:** Took 11 minutes to partially solve gates; succeeded in 2/10 attempts without internet, 6/10 with internet [20] - **Agent Performance:** Browser Use agents solved all gates within 2-5 minutes; MCP agents solved within 17 seconds [20] - **Hallucination Rate:** ~0.01% (4 words out of 4000, 2 domains out of 2000) [20]

Relevant Dimensions: - **(2) Cost-Aware Design:** Core contribution—imposing tunable costs on agents through computationally asymmetric challenges - **(3) Routing/Model Selection:** Dynamically controls challenge difficulty and selects challenges from pre-generated bank - **(6) Small Models $\leq 10B$:** Smaller models (o3-mini, Gemini 2.5-Flash) used as primary challenge generators - **(7) Benchmark Methodology:** Proposes new scalable and robust challenge generation framework (rRGs)

Actionable Insights for P79: - **Cost-Aware Design:** Throttling’s focus on computational asymmetry (9.2x cost difference) highlights the importance of measuring token consumption accurately. For P79, track token consumption separately for different modules (M1-M4) to identify optimization opportunities. - **Difficulty Control:** Throttling’s dynamic difficulty control provides a template for P79’s adaptive routing. Implement routing rules that adjust module selection based on task difficulty (e.g., easy tasks use simple representation, hard tasks use two-stage reasoning). - **Benchmark Methodology:** Throttling’s rebus-based reasoning gates (rRG) demonstrate the value of structured challenge generation. For P79, consider creating a difficulty-stratified subset of VisualWebArena tasks to evaluate routing effectiveness across difficulty levels. - **Limited Direct Applicability:** Throttling focuses on deterring malicious agents, which is orthogonal to P79’s goal of optimizing legitimate agent performance. However, the cost-awareness principles are transferable.

Limitations/Caveats: - **Operational Cost for Small Providers:** Reasoning Gates’ operational cost is non-trivial for small-scale resource providers, limiting viable deployment to larger organizations [20] - **Current Generation Costs:** Higher than desired, potentially reducing incentive for large-scale deployment [20] - **Adaptive Adversaries:** Impact of high-resource adversaries remains open question [20] - **Scalability and User Experience:** Further study needed on how difficulty settings affect latency and user experience at scale [20] - **Environmental Impact:** Framework increases LM consumption on both client and provider sides, contributing to environmental concerns [20]

Paper 21: GUIDE

Citation: Not available due to PDF parsing failure [21].

Note: PDF parsing failed for this paper. No detailed analysis possible.

Core Contribution: Unknown due to PDF parsing failure.

Key Quantitative Results: Not available.

Relevant Dimensions: Unknown.

Actionable Insights for P79: None available due to lack of information.

Limitations/Caveats: PDF parsing failed; comprehensive analysis not possible.

Paper 22: WebGraphEval

Citation: Yaoyao Qian, Yuanli Wang, Jinda Zhang, Yun Zong, Meixu Chen, Hanhan Zhou, Jindan Huang, Yifan Zeng, Xinyu Hu, Chan Hee Song, Danqing Zhang. “Web-GraphEval: Multi-Turn Trajectory Evaluation for Web Agents using Graph Representation.” NeurIPS 2025 Workshop: Multi-Turn Interactions in Large Language Models, 2025 [22].

Core Contribution: WebGraphEval presents a framework that abstracts trajectories from multiple agents into a unified, weighted action graph, providing a principled basis for analyzing solution spaces without modifying environments. The framework canonically encodes actions, merges recurring behaviors, and applies structural analyses including reward propagation and success-weighted edge statistics to capture cross-model regularities, highlight redundancy and inefficiency, and identify critical decision points [22].

Key Quantitative Results: - **Dataset Statistics (WebArena-based):** 4,768 total trajectories, 6 agent frameworks, 812 unique tasks, 40,431 total graph nodes, 45,656 total graph edges, 49.79 average nodes per task, 56.23 average edges per task [22] - **Total Individual Actions:** 40,888 (Clicks: 19,380 [47.4%], Type: 8,312 [20.3%], Select: 1,302 [3.2%]) [22] - **Successful Trajectories:** 2,180 (45.7%); Failed: 2,588 (54.3%) [22] - **Agent Performance:** Success rates range from 27.97% (OpenAI-CUA) to 64.78% (Jace.AI); average trajectory lengths range from 5.3 steps (IBM CUGA) to 15.6 steps (BrowserUse) [22] - **Necessity Rates:** Range from 72.9% (Learn by Interact) to 82.0% (UI-TARS); overall necessity rate: 76.7% of actions labeled necessary [22] - **Annotation Quality:** Canonicalization agreement with humans: 91% (376/411); Necessity judgment agreement: 78% (404/520) [22]

Relevant Dimensions: - **(1) Observation Representation:** Graph-based representation of action sequences - **(7) Benchmark Methodology:** Core contribution—novel evaluation framework for web agents

Actionable Insights for P79: - **Benchmark Methodology:** WebGraphEval’s graph-based trajectory analysis provides a sophisticated evaluation framework for P79. After collecting VisualWebArena trajectories, apply WebGraphEval’s methods to: 1. Identify redundant/exploratory actions (23.3% of actions in WebGraphEval dataset) 2. Detect critical decision points (bottlenecks, traps) 3. Compare routing strategies by analyzing graph structure - **Necessity Rate:** WebGraphEval’s finding that 76.7% of actions are necessary (23.3% redundant/exploratory) provides a target for P79’s efficiency optimization. Aim to reduce redundant actions through better routing and memory management. - **Action Distribution:** WebGraphEval’s action distribution (47.4% clicks, 20.3% type, 3.2% select) informs P79’s understanding of VisualWebArena task composition. Expect similar distribution, with clicks dominating. -

Success Rate Range: WebGraphEval reports 27.97%-64.78% success rates across 6 agent frameworks on WebArena, providing a reference range for P79’s VisualWebArena expectations (likely 15-40% for Qwen3-VL-4B).

Limitations/Caveats: - **Trajectory Availability Dependence:** Reliability of consensus graphs depends on diverse trajectory availability; tasks with few attempts or limited agent coverage yield less stable structural insights [22] - **Canonicalization Limitations:** Current state and action canonicalization implemented through heuristics and LLM-based prompts may struggle with breadth of real-world interfaces and actions [22] - **Contextual Completeness Constraints:** Many trajectories lack full screenshots or auxiliary information, limiting fidelity of environment reconstruction [22] - **Dataset Constraints:** Limited by what is captured in the dataset [22]

Paper 23: WebCanvas

Citation: Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, Zhengyang Wu. “WebCanvas: Benchmarking Web Agents in Online Environments.” arXiv:2406.12373v3 [cs.CL], 2024 [23].

Core Contribution: WebCanvas introduces an innovative online evaluation framework designed to address the dynamic nature of web interactions for benchmarking web agents. Its novelty lies in three main components: a novel evaluation metric that captures critical intermediate actions while disregarding noise, a refined benchmark dataset called Mind2Web-Live, and lightweight annotation tools and testing pipelines for community-driven data collection and maintenance [23].

Key Quantitative Results: - **Task Success Rate (Mind2Web-Live):** Best-performing agent achieved 23.1% task success rate [23] - **Task Completion Rate:** Best-performing agent achieved 48.8% task completion rate [23] - **Mind2Web-Live Dataset Size:** 542 high-quality tasks (438 training, 104 test), 2439 key nodes, 4550 detailed annotation steps [23] - **Mind2Web Expired Tasks:** 96 out of 780 tasks (12%) completely expired on live websites [23] - **GPT-4 Performance (Mind2Web-Live Test):** Completion Rate: 48.8% (± 3.04), Task SR: 23.1% (± 1.11), Efficiency Score: 2.47 (± 0.28) [23] - **GPT-3.5 Performance:** Completion Rate: 40.2% (± 0.38), Task SR: 16.5% (± 2.00), Efficiency Score: 3.03 (± 0.28) [23] - **MindAct Performance (Online vs. Offline):** Offline Task SR(0): 10.0%, Online: 7.50%; Offline Task SR(1): 25.0%, Online: 12.5% [23]

Relevant Dimensions: - **(1) Observation Representation:** Accessibility tree-based approach + screenshots for visual context - **(2) Cost-Aware Design:** Aims for cost-effective maintenance through scheduled monitoring and automated alerts - **(3) Routing/Model Selection:** Implies selection mechanism through “key nodes” to guide agent behavior - **(4) Memory Management:** Incorporates ‘Memory’ module to store historical information - **(7) Benchmark Methodology:** Core contribution—novel online evaluation framework and Mind2Web-Live dataset - **(8) Task Planning/Decomposition:** ‘Planning’ module integrates past action history, current observations, task instructions

Actionable Insights for P79: - **Benchmark Methodology:** WebCanvas’s key node concept (essential milestones that any task process must traverse) provides a template for P79’s evaluation methodology. For VisualWebArena, identify key nodes for each task (e.g., “Reached product page”, “Applied filter”, “Added to cart”) and evaluate success based on key node coverage, not just final outcome. - **Online vs. Offline Performance Gap:** WebCanvas reports significant performance degradation from offline to online evaluation (e.g., MindAct: 25.0% → 12.5% Task SR). For P79, expect similar degradation when deploying on live VisualWebArena; validate routing strategies on both static and dynamic versions. - **Memory Module:** WebCanvas’s ablation shows that less capable models (GPT-3.5) benefit from memory (+5.6% Task SR), while more capable models (GPT-4) show negative impact (-1.0% Task SR). For P79’s Qwen3-VL-4B (small model), expect memory module (M4) to provide positive contribution. - **Efficiency Score:** WebCanvas introduces efficiency score (average steps per successful task). For P79, track efficiency alongside success rate to measure routing effectiveness (good routing should improve both).

Limitations/Caveats: - **Network Instability:** Variability in network conditions (CAPTCHAs, network outages, IP inconsistencies) leads to discrepancies between online and closed environment results [23] - **Complex Task Pathways:** Diversity of execution paths may not be completely identified by annotators; misalignment between key nodes and essential components can inadvertently penalize correct processes [23] - **Static Evaluation Functions:** Cannot accommodate changes based on environmental variables (time, location, weather); needs dynamic logic/code-based reward system [23] - **Local Optima:** Agents get stuck due to multiple constraints, similar elements, lack of proactive thinking to revert to intermediate states [23] - **Premature Termination:** Agents partially complete tasks due to hallucination or failing to deliver correct actions [23] - **Information Loss in Observation:** Complex relationships between web elements; essential info may be in parent/child/sibling elements, leading to discarded elements if recursive search fails [23]

Paper 24: MoMA (Mixture of Models and Agents) / Generalized Routing

Citation: Xiyu Guo, Shan Wang, Chunfang Ji, Xuefeng Zhao, Wenhao Xi, Yaoyao Liu, Qinglan Li, Chao Deng, Junlan Feng. “Towards Generalized Routing: Model and Agent Orchestration for Adaptive and Efficient Inference.” arXiv:2509.07571v2 [cs.MA], 2025 [24].

Core Contribution: MoMA proposes a generalized routing framework that integrates both LLM-based and agent-based routing to handle diverse queries through precise intent recognition and adaptive routing strategies. Built upon detailed profiling of model and agent capabilities, the framework dynamically routes queries to the LLM with the best cost-performance efficiency during inference, and introduces an efficient agent selection strategy based on a context-aware state machine and dynamic masking [24].

Key Quantitative Results: - **Performance vs. Single LLMs:** Best single model (qwen3-235b-a22b): 68.6 across three benchmarks; Deepseek-r1: 60.2 [24] - **MoMA Performance-Priority Mode:** SOTA on AIME2024 and SimpleQA benchmarks; 2.9%

score improvement over optimal single model; 31.46% cost reduction [24] - **MoMA Auto-Routing Mode:** 37.19% cost reduction compared to performance-priority mode; achieves relatively high score (surpassing deepseek-v3) at significantly lower cost [24] - **Training Dataset:** ~2.25 million instances across science, writing, technology, programming domains [24] - **Model Architecture:** Base encoder: Qwen-3 (instruction-tuned LLM); top-k experts selected dynamically by gating network [24]

Relevant Dimensions: - **(1) Observation Representation:** Query encoding via pre-trained LLM hidden states - **(2) Cost-Aware Design:** Core contribution—Pareto optimization balancing performance and cost - **(3) Routing/Model Selection:** Core contribution—unified LLM and agent routing - **(4) Memory Management:** High-performance caching strategy - **(6) Small Models $\leq 10B$:** Explores performance across parameter scales, particularly smaller models - **(7) Benchmark Methodology:** Evaluates on AIME2024, SimpleQA, and other public benchmarks - **(8) Task Planning/Decomposition:** Agent clustering and state machine for complex task handling

Actionable Insights for P79: - **Phase 2 (Rule-based Routing):** MoMA’s two-layer routing (LLM routing + agent routing) provides a comprehensive template for P79’s routing architecture. Implement: 1. **Layer 1 (Model Selection):** Route to appropriate model size (e.g., Qwen3-VL-4B vs. 8B vs. 32B) based on task difficulty 2. **Layer 2 (Module Selection):** Within selected model, route to appropriate modules (M1-M4) based on current state - **Cost-Aware Design:** MoMA’s three preference modes (performance-priority, cost-priority, automatic routing) map to P79’s routing objectives. Implement automatic routing using TOPSIS-like algorithm that balances success rate and token consumption. - **Module M4 (Memory Management):** MoMA’s high-performance caching (cache key: standardized queries or semantic embeddings; cache value: final list of AI agents) provides a template for P79’s memory module. Cache successful trajectories and retrieve based on semantic similarity to current task. - **Pareto Frontier:** MoMA’s Pareto frontier concept (balancing costs and performance scores) is directly applicable to P79. For each routing decision, identify Pareto-optimal choices (e.g., DOM representation: low cost, medium performance; SoM: medium cost, high performance; two-stage reasoning: high cost, highest performance).

Limitations/Caveats: - **Agent Selection Challenges:** Existing LLM-based techniques struggle with precise and reliable selection in large-scale agent repositories, leaving room for improvement [24] - **Training Data Cost:** Main limitation of preference-based approach lies in high cost of constructing training data, as reliable preference labels depend on availability of strong judge model [24] - **Scalability under Diversity:** SFT-based approaches perform well under limited category conditions but degrade significantly in practical applications involving numerous categories [24] - MoMA is a learned router requiring training data; P79’s rule-based approach must approximate these learned patterns with heuristics

3. Cross-Paper Synthesis

This section synthesizes findings across all 24 papers to identify consensus, contradictions, and emerging patterns for each of the 8 research dimensions.

Dimension 1: Observation Representation

Consensus: - Accessibility Tree (A11y) / Set-of-Marks (SoM) for Small Models: Strong consensus that compact representations outperform full HTML for models $\leq 10\text{B}$ parameters [2], [6], [9], [15]. Read More, Think More demonstrates that gpt-oss-20b with HTML decreased to 27.6% (18.8pp worse than a11y) [2]. WebWorld adopts A11y Tree as primary representation due to universal applicability, high information density, and LLM-friendly structure [9].

- **HTML for Large Models:** Higher-capability models ($\geq 70\text{B}$ parameters) benefit from detailed HTML observations [2]. gpt-5.1 with HTML achieved 73.3% success rate, 17.5pp improvement over a11y [2].
- **Hybrid Multimodal (Screenshot + Text):** Strong consensus that combining visual and textual modalities improves performance [5], [14], [17], [19], [23]. Ego2Web’s ablation shows dramatic performance hierarchy: No Visual (4.4%) < Caption (23.6%) < Raw Video (48.2%) [17]. DMAST demonstrates that dual-modality attacks are more effective than single-modality, highlighting the importance of cross-modal consistency [5].
- **Semantic Element Descriptions:** Critical for cross-website generalization [6], [10], [15]. HMT’s catastrophic failure with raw element identifiers (39.7% $\rightarrow$ 12.4% StepSR) demonstrates the necessity of semantic descriptions (role, label, relative position, structural context) [6].

Contradictions: - Observation History: Mixed results on whether observation history improves performance. Read More, Think More shows consistent improvements (gpt-5.1: +3.0pp with hist4; gemini-2.5-flash: +10.9pp with hist4) [2]. However, WebCanvas reports that less capable models don’t consistently benefit from memory (GPT-4 with memory: -1.0% Task SR) [23].

Emerging Patterns: - Diff-based History: Emerging consensus that diff-based observation history offers comparable performance to full history while reducing tokens to $\sim 1/3$ [2]. This provides a token-efficient alternative for long-horizon tasks.

- **DOM Filtering:** WEBSERV’s approach of automatically reducing page to visible and meaningful elements (removing invisible/irrelevant nodes like `<script>`, `<style>`, hidden elements) is gaining traction [15].

Implications for P79: - Phase 1 (Representation Comparison): Test DOM (full HTML), SoM (accessibility tree), and hybrid (screenshot + SoM). Predict SoM will outperform DOM for Qwen3-VL-4B based on strong consensus for small models. - **Module M4 (Memory Management):** Implement diff-based observation history (hist4) to balance context and efficiency. - **Hybrid Representation:** Ensure high-quality screenshots are provided alongside SoM; don’t over-compress visual input to save tokens.

Dimension 2: Cost-Aware Design

Consensus: - Token Consumption Dominates Cost: Strong consensus that prompt tokens contribute >70% of total cost [4], [7]. WebRouter reports prompt tokens contribute over 70% of total price for all models [4].

- **Selective Compute Allocation:** Overwhelming consensus that selective application of expensive operations is more efficient than uniform scaling [7], [8], [24]. AVR achieves 78% cost reduction through selective routing [7]. CATTs achieves 56% token reduction with margin-gated confidence-aware scaling [8]. MoMA achieves 37.19% cost reduction with auto-routing [24].
- **Memory Compression:** Strong consensus that memory compression significantly reduces token consumption [1], [3], [6]. M² achieves 58.7% token reduction through dual-tier memory [3]. HMT achieves 71.0% cost reduction through hierarchical memory [6]. AgentSwing achieves up to 3x reduction in interaction turns [1].

Quantitative Targets: - Cost Reduction: Papers report 31.46%-87.8% cost reduction through routing [4], [7], [24] - **Token Reduction:** Papers report 40%-72.7% token reduction through memory management [3], [6] - **Accuracy Trade-off:** Most papers maintain accuracy within 2-5pp while achieving substantial cost reductions [4], [7], [8]

Emerging Patterns: - Pareto Optimization: Emerging framework for balancing cost and performance [24]. MoMA's Pareto frontier concept ($M = \{(m_k, c_k, s_k)\}$) provides a principled approach to routing decisions.

- **Difficulty-Adaptive Routing:** Emerging consensus that routing decisions should adapt to task difficulty [7], [20]. AVR's difficulty classification pre-routes easy actions to small model, hard actions to large model [7].

Implications for P79: - Cost Reduction Target: Aim for 40-70% token reduction through combined routing and memory management, maintaining success rate within 5pp of baseline. - **Phase 2 (Routing):** Implement difficulty-adaptive routing that applies expensive modules (M3, M4) only when task difficulty is high or confidence is low. - **Tracking:** Separately track prompt tokens and completion tokens to identify optimization opportunities.

Dimension 3: Routing/Model Selection

Consensus: - Confidence-Based Routing: Strong consensus that routing decisions should be based on model confidence [7], [8]. AVR uses logprobs to measure confidence; if below threshold (e.g., 0.85), escalate to larger model [7]. CATTs uses vote-derived uncertainty (entropy, margin) to decide when to apply additional compute [8].

- **Multi-Level Routing:** Emerging consensus on hierarchical routing architectures [11], [18], [24]. OpAgent uses modular framework (Planner, Grounder, Reflector, Summarizer) [11]. MoMA uses two-layer routing (LLM routing + agent routing) [24].
- **Lookahead Mechanisms:** AgentSwing demonstrates that moderate lookahead ($k=3$) is most effective for routing decisions [1]. Larger lookahead ($k=5$) doesn't always improve performance.

Contradictions: - Query-Level vs. Step-Level Routing: Papers diverge on routing granularity. WebRouter and MoMA focus on query-level routing (one decision per task) [4], [24], while AVR and CATTS focus on step-level routing (decision per action) [7], [8]. For web agents, step-level routing appears more effective.

Emerging Patterns: - Hybrid Routing Strategies: Combining multiple routing mechanisms (difficulty classification + confidence-based + safety-integrated) yields best results [7]. AVR's three-mechanism approach achieves 78% cost reduction [7].

- **Adaptive Thresholds:** Emerging consensus that routing thresholds should adapt to context [7], [8]. AVR uses difficulty-adaptive thresholds; CATTS uses vote-derived uncertainty thresholds.

Implications for P79: - Phase 2 (Routing): Implement three-mechanism routing: 1. **Difficulty Classification:** Use heuristics (form density, number of interactive elements, action history length) 2. **Confidence-Based Routing:** Use Qwen3-VL-4B's logprobs; if below 0.85, trigger M3 or M2 3. **Safety-Integrated Routing:** For high-risk actions (final submission), always use two-stage reasoning - **Lookahead:** Consider $k=3$ lookahead for memory management decisions (which context management strategy to use).

Dimension 4: Memory Management

Consensus: - Hierarchical Memory Structures: Strong consensus that hierarchical memory outperforms flat memory [6], [13]. HMT's three-level hierarchy (Intent, Stage, Action) provides 6.6% gain over flat memory [6]. StructuredAgent's AND/OR tree planning achieves 52.6% overall score on WebArena [13].

- **Dual-Tier Memory:** Emerging consensus on separating short-term and long-term memory [3], [19]. M2's dual-tier approach (Internal Memory for recent events, External Memory for historical insights) achieves 58.7% token reduction [3]. Egocentric Co-Pilot's T-CoT + HCC achieves strong performance with Qwen2.5-VL-7B [19].
- **Semantic Compression:** Strong consensus that semantic compression (summarization, abstraction) is more effective than simple truncation [1], [3], [6]. AgentSwing's Summary strategy outperforms Discard-All on some benchmarks [1].

Quantitative Results: - Token Reduction: Hierarchical memory achieves 58.7%-72.7% token reduction [3], [6] - **Performance Improvement:** Hierarchical memory

provides 5-10pp improvement over flat memory [6], [13] - **Recall Improvement:** HMT achieves 84.2% recall vs. flat retrieval’s 65.8% [6]

Emerging Patterns: - **Skill Libraries:** Emerging concept of storing reusable skills/patterns as structured memory [10], [12]. ContractSkill achieves 47.8pp average gain on VWA through cross-model skill transfer [10]. SkillWeaver’s skill library improves success rate from 65% to 82% after refinement [12].

- **Pre/Post-Conditions:** Emerging consensus that explicit state tracking (preconditions, postconditions) improves memory effectiveness [6], [10], [13]. HMT’s ablation shows 2.5% drop without pre/post-conditions [6].

Implications for P79: - **Module M4 (Memory Management):** Implement three-level hierarchical memory: 1. **Intent Level:** Task goal (e.g., “Find product with constraints”) 2. **Stage Level:** Functional subgoals (e.g., “Navigate to category”, “Apply filters”, “Select item”) 3. **Action Level:** Specific actions with semantic element descriptions - **Dual-Tier Approach:** Separate short-term memory (last 4-5 steps with full detail) from long-term memory (compressed summaries of older history). - **Skill Library:** Pre-compute 50-100 successful VisualWebArena trajectories, retrieve top-5 similar cases when confidence is low.

Dimension 5: Grounding/Failure Recovery

Consensus: - **Deterministic Verification:** Strong consensus that explicit verification of expected state changes improves reliability [10], [15]. ContractSkill’s deterministic verification (checking URL, DOM state, form values) enables fault localization [10]. WEBSERV intercepts network events and waits for page quiescence before returning observations [15].

- **Fault Localization:** Emerging consensus that identifying where failures occur enables more effective recovery [10], [11]. ContractSkill’s fault localization provides major benefit (w/o localization: GLM 65.0%, Qwen 70.0% vs. full 77.5%, 81.0%) [10]. OpAgent’s Reflector analyzes why actions failed [11].
- **Local Repair:** Emerging consensus that localized repair is more efficient than full rewriting [10]. ContractSkill’s constrained local repair outperforms unconstrained editing (GLM 68.5% vs. 77.5%) [10].

Contradictions: - **Retry Strategies:** Papers diverge on optimal retry strategies. Some advocate for naive retry with same approach [8], others for adaptive retry with different strategies [5], [10]. DMAST’s adversarial RL demonstrates that adaptive retry is more effective [5].

Emerging Patterns: - **Semantic Grounding:** Emerging consensus that semantic element descriptions (role, label, relative position) are critical for robust grounding [6], [10], [15]. HMT’s catastrophic failure with raw identifiers demonstrates this [6].

- **Interactive Clarification:** Emerging concept of detecting uncertainty and issuing clarification questions [19]. Egocentric Co-Pilot’s clarification module detects semantic uncertainty and requests additional information [19].

Implications for P79: - Module M1 (Retry): Implement adaptive retry with escalating strategies: 1. First retry: Same representation, different action 2. Second retry: Different representation (DOM→SoM) 3. Third retry: Two-stage reasoning (M3) - **Module M2 (Fallback):** Implement fault localization to identify failure type (element not found, action had no effect, wrong element selected), then apply appropriate fallback strategy. - **Verification:** After each action, verify expected state change (URL changed, element appeared, form value updated).

Dimension 6: Small Models $\leq 10B$

Consensus: - Compact Representations Preferred: Strong consensus that small models perform better with compact representations (a11y, SoM) than full HTML [2], [9]. Read More, Think More shows gpt-oss-20b with HTML decreased to 27.6% (18.8pp worse than a11y) [2].

- **Memory Benefits Small Models:** Emerging consensus that memory modules provide larger benefits for small models than large models [3], [23]. M² enables Qwen3-VL-32B to match proprietary performance [3]. WebCanvas shows GPT-3.5 benefits from memory (+5.6% Task SR) while GPT-4 shows negative impact (-1.0%) [23].
- **Reasoning Activation:** WebWorld demonstrates that minimal CoT data (1,000 samples) activates reasoning capabilities in small models [9]. This suggests that small models can perform two-stage reasoning with proper prompting.

Quantitative Baselines: - VisualWebArena: ContractSkill reports 56.5%-58.0% baseline success rates on VWA for models in 4B-6B range [10]. DMAST reports 6.2% TSR for base Gemma-3-12B-IT on VWA [5]. - **WebArena:** OpAgent’s RL-enhanced model (pass@5) achieved 38.1% success rate [11]. - **WebVoyager:** Web-CogReasoner (Qwen2.5-VL-7B) achieves 30.2% [14].

Emerging Patterns: - Modular Architectures: Emerging consensus that small models benefit from modular architectures that decompose complex tasks [11], [13], [19]. OpAgent’s modular framework (Planner, Grounder, Reflector, Summarizer) achieves strong performance [11]. Egocentric Co-Pilot achieves 4.70/5.0 rating with Qwen2.5-VL-7B [19].

Implications for P79: - Baseline Expectations: For Qwen3-VL-4B on VisualWebArena, expect baseline success rate of 10-30% based on similar-sized models’ performance. - **Representation:** Prioritize SoM representation over DOM for Qwen3-VL-4B. - **Memory:** Expect M4 to provide substantial benefit (10-20pp improvement) for Qwen3-VL-4B. - **Two-Stage Reasoning:** Implement M3 with minimal CoT prompting (no fine-tuning required).

Dimension 7: Benchmark Methodology

Consensus: - Online vs. Offline Gap: Strong consensus that online evaluation reveals significant performance degradation compared to offline [23]. WebCanvas reports MindAct performance drop from 25.0% (offline) to 12.5% (online) Task SR [23]. Mind2Web had 12% of tasks completely expired on live websites [23].

- **Key Node Evaluation:** Emerging consensus that evaluating intermediate milestones (key nodes) is more informative than final outcome alone [23]. WebCanvas’s key node concept captures critical intermediate actions while disregarding noise [23].
- **LLM-as-a-Judge:** Emerging consensus that LLM-based evaluation can approximate human judgment with 78-84% agreement [17], [23]. Ego2Web’s Ego2WebJudge achieves 84% agreement with humans [17].

Quantitative Benchmarks: - WebArena: Success rates range from 27.97% (OpenAI-CUA) to 71.6% (OpAgent SOTA) [11], [22] - **VisualWebArena:** Success rates range from 6.2% (base Gemma-3-12B-IT) to 81.0% (ContractSkill with Qwen3.5-Plus) [5], [10] - **WebVoyager:** Success rates range from 30.2% (Web-CogReasoner Qwen2.5-VL-7B) to 54.9% (Gemini 2.5 Pro) [14]

Emerging Patterns: - Graph-Based Analysis: WebGraphEval’s graph-based trajectory analysis provides sophisticated evaluation framework [22]. Identifies redundant actions (23.3%), critical decision points, and cross-model regularities [22].

- **Multi-Dimensional Evaluation:** Emerging consensus on evaluating multiple dimensions (success rate, efficiency, necessity rate, token consumption) [22], [23]. WebCanvas introduces efficiency score (average steps per successful task) [23].

Implications for P79: - Evaluation Methodology: Adopt multi-dimensional evaluation: 1. **Success Rate:** Final task completion 2. **Key Node Coverage:** Percentage of critical milestones reached 3. **Efficiency:** Average steps per successful task 4. **Token Consumption:** Total tokens used per task 5. **Necessity Rate:** Percentage of actions that are necessary (not redundant) - **Baseline Validation:** Validate routing strategies on both static and dynamic versions of VisualWebArena to account for online vs. offline gap. - **LLM-as-a-Judge:** Consider using LLM-based evaluation for rapid iteration, but validate against human judgment on subset.

Dimension 8: Task Planning/Decomposition

Consensus: - Hierarchical Planning: Strong consensus that hierarchical planning outperforms flat planning [6], [13], [14]. StructuredAgent’s AND/OR tree planning achieves 52.6% overall score on WebArena [13]. HMT’s three-level hierarchy (Intent, Stage, Action) improves StepSR by 6.0% [6].

- **Knowledge-Driven Decomposition:** Emerging consensus that decomposing tasks into knowledge layers (Factual, Conceptual, Procedural) improves perfor-

mance [14]. Web-CogReasoner’s knowledge-driven CoT achieves 84.4% on Web-CogBench [14].

- **Skill Composition:** Emerging consensus that composing reusable skills is more efficient than monolithic planning [10], [12]. ContractSkill achieves 47.8pp average gain through skill transfer [10]. SkillWeaver improves success rate from 65% to 82% through skill refinement [12].

Quantitative Results: - **Hierarchical Planning Improvement:** 5-10pp improvement over flat planning [6], [13] - **Knowledge-Driven CoT:** +17.9% (Factual), +11.3% (Conceptual), +19.2% (Procedural) cumulative gains [14] - **Skill Refinement:** 65% → 82% success rate after 3 refinement iterations [12]

Emerging Patterns: - **AND/OR Trees:** StructuredAgent’s AND/OR tree planning provides explicit representation of alternative paths [13]. This enables systematic backtracking and exploration of alternative branches.

- **Pre/Post-Conditions:** Explicit state tracking through preconditions and post-conditions improves planning reliability [6], [10]. HMT’s ablation shows 2.5% drop without pre/post-conditions [6].

Implications for P79: - **Module M3 (Two-Stage Reasoning):** Implement hierarchical planning: 1. **Stage 1 (Planning):** Decompose task into AND/OR tree or knowledge layers (Factual, Conceptual, Procedural) 2. **Stage 2 (Execution):** Execute leaf nodes, backtrack on failure, try alternative branches - **Skill Library:** Pre-compute common task patterns (e.g., “Search for product”, “Apply price filter”, “Add to cart”) and store as reusable skills with pre/post-conditions. - **Phase 3 (Ablation):** Test M3 with and without hierarchical planning to measure marginal contribution.

4. Gaps in Literature

This section identifies aspects of P79’s research question that are NOT well-covered in the current literature.

Gap 1: Step-Level Adaptive Routing for Small VLMs on Complex Benchmarks

What’s Missing: - Most routing papers focus on query-level routing (one decision per task) [4], [24] or use large models ($\geq 30\text{B}$ parameters) [1], [7] - No papers specifically investigate step-level adaptive routing for small VLMs ($\leq 10\text{B}$ parameters) on complex benchmarks like VisualWebArena - AVR comes closest but focuses on Computer Use Agents (CUAs) with different task characteristics [7]

Why It Matters for P79: - P79’s core research question is precisely this gap: how to optimize step-level routing for Qwen3-VL-4B on VisualWebArena - Existing routing strategies may not transfer directly to small VLMs due to different confidence calibration, reasoning capabilities, and context limitations

Specific Unknowns: - Optimal routing thresholds for Qwen3-VL-4B (e.g., confidence threshold for triggering two-stage reasoning) - Interaction between routing decisions

and observation modality (DOM/SoM/hybrid) for small VLMs - Trade-off between routing overhead (probing, confidence estimation) and routing benefits for small VLMs

Gap 2: Interaction Between Observation Modality and Failure Recovery for VLMs

What's Missing: - Read More, Think More investigates observation representation [2], ContractSkill investigates failure recovery [10], but no papers systematically study their interaction - Specifically: How does choice of observation modality (DOM/SoM/hybrid) affect optimal retry and fallback strategies? - DMAST studies cross-modal attacks [5] but doesn't investigate how to leverage multiple modalities for failure recovery

Why It Matters for P79: - P79's Phase 1 (representation comparison) and Modules M1/M2 (retry/fallback) are tightly coupled - Optimal retry strategy may differ for DOM vs. SoM representations (e.g., DOM retry might benefit from switching to SoM, but SoM retry might not benefit from switching to DOM)

Specific Unknowns: - Should retry mechanism switch observation modality or keep same modality? - Does hybrid representation (screenshot + DOM/SoM) enable more effective failure recovery than single modality? - How to detect which modality is causing failure (visual grounding error vs. textual grounding error)?

Gap 3: Memory Management for Small VLMs on Long-Horizon Visual Tasks

What's Missing: - M² and HMT demonstrate effective memory management [3], [6], but both focus on text-heavy tasks (WebVoyager, Mind2Web) rather than visual tasks (VisualWebArena) - No papers specifically investigate how to manage visual memory (screenshots) for small VLMs with limited context - Egocentric Co-Pilot uses video at 1 FPS [19], but this is for egocentric video, not web navigation

Why It Matters for P79: - VisualWebArena tasks require visual understanding (e.g., product images, layout, visual feedback) - Storing full screenshots in memory quickly exhausts context for small VLMs - Optimal balance between visual memory (screenshots) and textual memory (DOM/SoM) is unknown

Specific Unknowns: - Should memory store screenshots for all steps, only key steps, or no screenshots (text-only)? - How to compress visual memory without losing task-critical information? - Does diff-based visual memory (storing only changed regions) work for web navigation? - Optimal memory window size for VisualWebArena tasks (hist4, hist9, or adaptive)?

Gap 4: Cost-Aware Module Ablation for Web Agents

What’s Missing: - Many papers propose individual modules (retry, fallback, two-stage reasoning, memory management) [1], [3], [6], [8], [10], [11], but no papers systematically ablate all modules together to measure marginal contributions and interactions - Specifically: What is the marginal contribution of each module when all others are present? Do modules interact synergistically or redundantly?

Why It Matters for P79: - P79’s Phase 3 (module ablation) is designed to answer this question, but literature provides limited guidance on expected results - Understanding module interactions is critical for cost-aware routing (e.g., if M3 and M4 are redundant, only use one)

Specific Unknowns: - Marginal contribution of M1 (retry) when M2 (fallback) is present - Marginal contribution of M3 (two-stage reasoning) when M4 (memory management) is present - Interaction between M1 and M2 (does retry reduce need for fallback, or vice versa?) - Interaction between M3 and M4 (does two-stage reasoning reduce need for memory, or vice versa?) - Optimal module combination for different task types (e.g., search tasks vs. form-filling tasks)

Gap 5: Benchmark-Specific Routing Strategies

What’s Missing: - Most routing papers evaluate on multiple benchmarks but don’t investigate benchmark-specific routing strategies [4], [7], [24] - Specifically: Do optimal routing strategies differ between WebArena, VisualWebArena, WebVoyager, Mind2Web? - No papers provide detailed analysis of task characteristics (e.g., form density, navigation depth, visual complexity) and their impact on routing effectiveness

Why It Matters for P79: - VisualWebArena has unique characteristics (visual-heavy, shopping-focused, long-horizon) that may require different routing strategies than other benchmarks - Understanding task characteristics enables more effective rule-based routing

Specific Unknowns: - Task characteristics that predict routing effectiveness (e.g., form density → two-stage reasoning beneficial) - Benchmark-specific optimal routing thresholds (e.g., confidence threshold for VisualWebArena vs. WebArena) - Generalization of routing strategies across benchmarks (e.g., does routing strategy learned on WebArena transfer to VisualWebArena?)

Gap 6: Small Model Confidence Calibration for Routing

What’s Missing: - AVR and CATTS use confidence-based routing [7], [8], but both focus on large models ($\geq 70\text{B}$ parameters) or don’t report confidence calibration quality - Specifically: Are small VLMs ($\leq 10\text{B}$ parameters) well-calibrated? Can their confidence scores be trusted for routing decisions? - No papers investigate how to improve confidence calibration for small VLMs

Why It Matters for P79: - P79's routing logic relies on Qwen3-VL-4B's confidence scores (logprobs) - If confidence is poorly calibrated (e.g., overconfident on incorrect actions), routing will be ineffective - May need to calibrate confidence scores or use alternative uncertainty metrics

Specific Unknowns: - Confidence calibration quality for Qwen3-VL-4B (Expected Calibration Error, Brier Score) - Correlation between confidence and actual correctness for small VLMs - Alternative uncertainty metrics for small VLMs (e.g., vote-derived uncertainty, semantic similarity to training data) - How to improve confidence calibration without fine-tuning (e.g., temperature scaling, Platt scaling)

Gap 7: Real-World Deployment Considerations

What's Missing: - Most papers evaluate on static benchmarks in controlled environments [1], [2], [3], [4], [6], [7], [8], [9], [10], [11], [12], [13], [14] - WebCanvas and Ego2Web investigate online evaluation [17], [23], but don't provide detailed guidance on deployment challenges - Specifically: How do routing strategies perform under network latency, rate limits, CAPTCHA, dynamic content?

Why It Matters for P79: - While P79 focuses on offline evaluation, understanding real-world deployment challenges informs routing design - Routing strategies that work well offline may fail online due to latency, rate limits, or dynamic content

Specific Unknowns: - Impact of network latency on routing effectiveness (e.g., does probing overhead negate routing benefits?) - Robustness of routing strategies to CAPTCHA, rate limits, authentication barriers - Adaptation of routing strategies to dynamic content (e.g., real-time price changes, stock availability) - Trade-off between routing complexity and deployment simplicity

5. Priority Recommendations for P79

This section provides the top 5 most actionable findings ranked by relevance to P79's experimental design.

Recommendation 1: Prioritize SoM Representation for Qwen3-VL-4B (Phase 1)

Ranking: #1 (Highest Priority)

Evidence: - Read More, Think More demonstrates that lower-capability models perform better with compact representations: gpt-oss-20b with HTML decreased to 27.6% (18.8pp worse than a11y) [2] - WebWorld adopts A11y Tree as primary representation due to universal applicability, high information density, and LLM-friendly structure [9] - WEBSERV's DOM parser approach (filtering invisible/irrelevant nodes, preserving key visual cues) achieves 46.7% success on WebArena-Lite [15]

Specific Action for P79: - Phase 1 (Representation Comparison): Test three representations: 1. **DOM:** Full HTML with minimal filtering 2. **SoM:** Accessibility tree with semantic element descriptions (role, label, visible text, relative position) 3. **Hybrid:** Screenshot + SoM (not Screenshot + DOM) - **Prediction:** SoM will outperform DOM by 10-20pp for Qwen3-VL-4B - **Implementation Details:** - For SoM, include: element ID, tag, role, visible text (truncated to 50 chars), bounding box, parent element context - For Hybrid, ensure screenshot resolution is sufficient for visual grounding (1080×720 minimum) - For DOM, filter out <script>, <style>, hidden elements, but retain structural information

Why This Matters: - Representation choice is foundational; all subsequent modules (M1-M4) depend on it - Wrong representation choice could doom entire experiment (e.g., if DOM overwhelms Qwen3-VL-4B’s context, routing won’t help) - Strong consensus in literature provides high confidence in this recommendation

Potential Risks: - SoM may lose critical visual layout information (e.g., grid arrangements, spatial relationships) - Hybrid representation may still overwhelm context for long-horizon tasks - Mitigation: If SoM underperforms, investigate whether specific task types (e.g., visual search) require Hybrid

Recommendation 2: Implement Three-Mechanism Routing (Phase 2)

Ranking: #2

Evidence: - AVR’s three-mechanism approach (difficulty classification + confidence-based routing + safety-integrated routing) achieves 78% cost reduction [7] - CATTS demonstrates that confidence-aware scaling (margin-gated) achieves 56% token reduction [8] - AgentSwing shows that moderate lookahead (k=3) is most effective for routing decisions [1]

Specific Action for P79: - Phase 2 (Rule-based Routing): Implement three routing mechanisms:

Mechanism 1: Difficulty Classification (Pre-Action) - Heuristics: - Form density: Count input/select elements on page; if >5, classify as “hard” - Navigation depth: Track steps from homepage; if >10, classify as “hard” - Action history length: If >15 actions, classify as “hard” - **Routing Rules:** - Easy tasks: Use SoM representation, no two-stage reasoning, minimal memory (hist2) - Medium tasks: Use SoM representation, selective two-stage reasoning (confidence-based), moderate memory (hist4) - Hard tasks: Use Hybrid representation, always two-stage reasoning, full memory (hist9 with compression)

Mechanism 2: Confidence-Based Routing (Post-Action) - Confidence Estimation: - Extract logprobs from Qwen3-VL-4B’s action prediction - Calculate confidence score: $\text{conf} = \exp(\text{logprob_top1})$ - Calculate margin: $\text{margin} = \text{logprob_top1} - \text{logprob_top2}$ - **Routing Rules:** - If $\text{conf} < 0.85$ OR $\text{margin} < 0.5$: Trigger M3 (two-stage reasoning) - If $\text{conf} < 0.70$ OR $\text{margin} < 0.3$: Trigger M2 (fallback to different representation) - If $\text{conf} < 0.50$: Trigger M1 (retry with memory retrieval)

Mechanism 3: Safety-Integrated Routing (Task-Specific) - High-Risk Actions:

- Final submission (e.g., “Place Order”, “Submit Form”) - Irreversible actions (e.g., “Delete”, “Confirm Purchase”) - Actions with high cost (e.g., “Add to Cart” for expensive items) - **Routing Rules:** - Always use two-stage reasoning for high-risk actions - Always verify expected state change before proceeding - If verification fails, trigger M2 (fallback)

Why This Matters: - Single-mechanism routing is insufficient; combining multiple mechanisms yields best results - Three mechanisms cover different failure modes: difficulty classification prevents overconfidence on hard tasks, confidence-based routing detects uncertainty, safety-integrated routing prevents catastrophic errors - Provides clear, implementable routing logic for P79’s rule-based approach

Potential Risks: - Routing overhead (probing, confidence estimation) may negate benefits for very short tasks - Thresholds (0.85, 0.70, 0.50) are estimates; may need tuning on VisualWebArena - Mitigation: Track routing overhead separately; if overhead >20% of total tokens, adjust thresholds

Recommendation 3: Implement Dual-Tier Hierarchical Memory (Module M4)

Ranking: #3

Evidence: - M² achieves 58.7% token reduction through dual-tier memory (Internal + External) [3] - HMT achieves 71.0% cost reduction through three-level hierarchical memory [6] - Egocentric Co-Pilot’s T-CoT + HCC achieves strong performance with Qwen2.5-VL-7B [19]

Specific Action for P79: - Module M4 (Memory Management): Implement dual-tier hierarchical memory:

Tier 1: Short-Term Memory (Internal Memory) - Content: Last 4-5 steps with full detail - **Format:** For each step, store: - Screenshot (if using Hybrid representation) - SoM representation (compressed to top-20 interactive elements) - Action taken (type, target element, arguments) - Result (success/failure, state change) - Reasoning (if two-stage reasoning was used) - **Compression:** Use diff-based representation for SoM (store only changed elements) - **Token Budget:** Allocate 30-40% of max context to short-term memory

Tier 2: Long-Term Memory (External Memory) - Content: Compressed summaries of older history + skill library - **Format:** - **Trajectory Summary:** Every 5 steps, generate summary: “Current state: [location], Actions taken: [list], Constraints satisfied: [list], Next goal: [goal]” - **Skill Library:** Pre-compute 50-100 successful VisualWebArena trajectories, extract common patterns (e.g., “Search for product”, “Apply price filter”, “Add to cart”) - **Retrieval:** When confidence is low (<0.70), retrieve top-5 similar trajectories via semantic similarity (cosine similarity using Sentence Transformer) - **Token Budget:** Allocate 10-20% of max context to long-term memory

Hierarchical Structure (Three Levels): - **Intent Level:** Task goal (e.g., “Find product with price <\$50, rating >4.0”) - **Stage Level:** Functional subgoals (e.g.,

“Navigate to category”, “Apply filters”, “Select item”, “Add to cart”) - **Action Level:** Specific actions with semantic element descriptions

Why This Matters: - Memory management is critical for long-horizon tasks (VisualWebArena tasks average 15-30 steps) - Dual-tier approach balances detail (short-term) and efficiency (long-term) - Hierarchical structure enables better retrieval and reasoning - Strong consensus in literature (M², HMT, Egocentric Co-Pilot) provides high confidence

Potential Risks: - Skill library requires successful trajectories; limited VisualWebArena training data may constrain library size - Retrieval latency may negate benefits (M² reports 6ms latency, but this is for text-only retrieval) - Compression may lose critical information - Mitigation: Start with small skill library (50 trajectories), expand if effective; track retrieval latency; validate compression quality on subset

Recommendation 4: Implement Adaptive Retry with Fault Localization (Modules M1 & M2)

Ranking: #4

Evidence: - ContractSkill’s fault localization provides major benefit: w/o localization (GLM 65.0%, Qwen 70.0%) vs. full (77.5%, 81.0%) [10] - DMAST’s adversarial RL demonstrates that adaptive retry is more effective than naive retry [5] - CATTS shows that semantic deduplication is crucial for effective vote aggregation [8]

Specific Action for P79: - Module M1 (Retry) & M2 (Fallback): Implement adaptive retry with fault localization:

Step 1: Deterministic Verification (After Each Action) - Verify Expected State Change: - URL changed (for navigation actions) - Element appeared/disappeared (for click actions) - Form value updated (for type actions) - Page content changed (for any action) - **Implementation:** Compare current state to expected state (defined in action prediction) - **Outcome:** If verification passes, proceed; if fails, trigger fault localization

Step 2: Fault Localization (On Verification Failure) - Identify Failure Type:
- **Element Not Found:** Target element ID not in current SoM - **Action Had No Effect:** State unchanged after action execution - **Wrong Element Selected:** Action executed on wrong element (detected via visual grounding mismatch) - **Page Load Timeout:** Page didn’t reach quiescence within timeout - **Implementation:** Use rule-based heuristics to classify failure type - **Outcome:** Route to appropriate retry/fallback strategy based on failure type

Step 3: Adaptive Retry/Fallback (Based on Failure Type) - Element Not Found:
- **Retry 1:** Refresh SoM representation (page may have updated) - **Retry 2:** Switch to Hybrid representation (element may be visible but not in SoM) - **Fallback:** Retrieve similar trajectory from skill library, try alternative action sequence - **Action Had No Effect:** - **Retry 1:** Wait 2 seconds, verify again (page may be loading) - **Retry 2:** Try alternative action (e.g., click different element with same semantic label) - **Fallback:** Trigger two-stage reasoning (M3) to re-plan - **Wrong Element Selected:**

- **Retry 1:** Use visual grounding to verify element before action - **Retry 2:** Switch to more specific element description (add parent context) - **Fallback:** Switch to Hybrid representation for better visual grounding - **Page Load Timeout:** - **Retry 1:** Increase timeout to 10 seconds - **Retry 2:** Refresh page, try action again - **Fallback:** Skip action, proceed to next step (may be non-critical)

Step 4: Semantic Deduplication (For Multiple Retries) - If sampling $N > 1$
actions: Deduplicate semantically similar actions before voting - **Implementation:** Calculate semantic similarity between action descriptions using Sentence Transformer; merge actions with similarity > 0.9 - **Outcome:** Avoid redundant computation on similar actions

Why This Matters: - Naive retry (same action, same representation) is ineffective; adaptive retry addresses root cause of failure - Fault localization enables targeted retry/fallback strategies - Semantic deduplication improves efficiency of multi-sample retry - Strong evidence from ContractSkill, DMAST, CATTS

Potential Risks: - Fault localization may misclassify failure type, leading to ineffective retry - Adaptive retry increases complexity and potential for cascading errors - Multiple retries increase token consumption - Mitigation: Limit retries to 3 per action; track retry effectiveness; if retry success rate $< 30\%$, disable adaptive retry

Recommendation 5: Implement Knowledge-Driven Two-Stage Reasoning (Module M3)

Ranking: #5

Evidence: - Web-CogReasoner's knowledge-driven CoT achieves 84.4% on Web-CogBench with cumulative gains: +17.9% (Factual), +11.3% (Conceptual), +19.2% (Procedural) [14] - StructuredAgent's AND/OR tree planning achieves 52.6% overall score on WebArena [13] - WebWorld demonstrates that minimal CoT data (1,000 samples) activates reasoning capabilities in small models [9]

Specific Action for P79: - Module M3 (Two-Stage Reasoning): Implement knowledge-driven two-stage reasoning:

Stage 1: Knowledge Retrieval & Planning - Prompt Template: ““ Task: {task_description} Current State: {current_state_summary} Observation: {current_SoM_representation}

Before selecting an action, let's think step-by-step:

1. Factual Knowledge: What do I know about this website/task?
 - Website structure: {retrieve_from_skill_library}
 - Element types: {identify_interactive_elements}
 - Task requirements: {parse_task_constraints}
2. Conceptual Knowledge: What strategy should I use?
 - Task type: {classify_task_type: search/filter/select/checkout}
 - Current stage: {identify_current_stage: navigation/filtering/selection}

- Next goal: {determine_next_subgoal}

3. Procedural Knowledge: How do I execute this strategy?

- Action sequence: {plan_action_sequence}
- Expected outcome: {predict_state_change}
- Verification: {define_success_criteria}

Based on this reasoning, the best action is:

```

- **Implementation:** Prompt Qwen3-VL-4B to generate explicit reasoning before action selection
- **Token Budget:** Allocate 200-300 tokens for reasoning (10-15% of typical action)

**Stage 2: Action Selection & Execution - Prompt Template:** “ Reasoning: {reasoning\_from\_stage1}

Now, select the specific action:

- Action Type: {click/type/select/scroll}
- Target Element: {element\_id\_from\_SoM}
- Arguments: {if\_applicable}
- Expected State Change: {from\_reasoning}

```

- **Implementation:** Use reasoning from Stage 1 to guide action selection
- **Verification:** After execution, verify that expected state change occurred (from Stage 1 reasoning)

Trigger Conditions (When to Use Two-Stage Reasoning): - **Always:** High-risk actions (final submission, irreversible actions) - **Confidence-Based:** When confidence < 0.85 OR margin < 0.5 - **Difficulty-Based:** When task classified as “hard” (form density > 5, navigation depth > 10) - **Failure-Based:** After retry/fallback (to re-plan)

Why This Matters: - Two-stage reasoning provides explicit reasoning trace, improving interpretability and debuggability - Knowledge-driven approach (Factual, Conceptual, Procedural) structures reasoning, reducing hallucination - Minimal CoT prompting (no fine-tuning) makes this feasible for Qwen3-VL-4B - Strong evidence from Web-CogReasoner, StructuredAgent, WebWorld

Potential Risks: - Two-stage reasoning increases token consumption (200-300 tokens per action) - May not improve performance if Qwen3-VL-4B’s reasoning capabilities are limited - Reasoning may be verbose or irrelevant - Mitigation: Use two-stage reasoning selectively (only when triggered); track token overhead; if overhead > 30% with < 5pp improvement, reduce usage

6. Conclusion

This literature review analyzed 24 recent papers (2024-2026) on web agents, providing a comprehensive foundation for Project P79’s investigation of cost-aware routing

for web usage agents. The review identified strong consensus on several key dimensions:

1. **Observation Representation:** Small models ($\leq 10B$ parameters) perform better with compact representations (accessibility tree, SoM) than full HTML, with hybrid multimodal approaches (screenshot + text) providing best results.
2. **Cost-Aware Design:** Selective compute allocation through routing achieves 40-80% cost reduction while maintaining accuracy within 2-5pp of baseline. Token consumption is dominated by prompt tokens ($> 70\%$), making memory compression critical.
3. **Routing/Model Selection:** Confidence-based routing with multiple mechanisms (difficulty classification, confidence estimation, safety integration) yields best results. Step-level routing is more effective than query-level routing for web agents.
4. **Memory Management:** Hierarchical memory structures (dual-tier, three-level) outperform flat memory, achieving 58-72% token reduction and 5-10pp performance improvement. Semantic compression and skill libraries are emerging as effective strategies.
5. **Grounding/Failure Recovery:** Deterministic verification, fault localization, and adaptive retry are critical for robust web agents. Semantic element descriptions are essential for cross-website generalization.

However, significant gaps remain in understanding optimal routing strategies for small VLMs ($\leq 10B$ parameters) on complex visual benchmarks like VisualWebArena, particularly regarding the interaction between observation modality, memory management, and failure recovery mechanisms. P79’s experimental design directly addresses these gaps through systematic investigation of representation comparison (Phase 1), rule-based routing (Phase 2), and module ablation (Phase 3).

The five priority recommendations provide concrete, actionable guidance for P79’s implementation: 1. Prioritize SoM representation for Qwen3-VL-4B 2. Implement three-mechanism routing (difficulty classification, confidence-based, safety-integrated) 3. Implement dual-tier hierarchical memory (short-term + long-term with skill library) 4. Implement adaptive retry with fault localization 5. Implement knowledge-driven two-stage reasoning

These recommendations are grounded in strong empirical evidence from the literature and tailored to P79’s specific experimental setup (Qwen3-VL-4B, VisualWebArena, modules M1-M4). By following these recommendations, P79 is well-positioned to make significant contributions to the understanding of cost-aware routing for web usage agents.

7. References

[1] Zhaopeng Feng, Liangcai Su, Zhen Zhang, Xinyu Wang, Xiaotian Zhang, Xiaobin Wang, Runnan Fang, Qi Zhang, Baixuan Li, Shihao Cai, Rui Ye, Hui

- Chen, Jiang Yong, Joey Tianyi Zhou, Chenxiong Qian, Pengjun Xie, Bryan Hooi, Zuozhu Liu, Jingren Zhou. "AGENTS WING: Adaptive Parallel Context Management Routing for Long-Horizon Web Agents." arXiv:2603.27490v1 [cs.CL], 2026. <https://doi.org/10.48550/arxiv.2603.27490>
- [2] Masafumi Enomoto, Ryoma Obara, Haochen Zhang, Masafumi Oyamada. "Read More, Think More: Revisiting Observation Reduction for Web Agents." arXiv:2604.01535v1 [cs.CL], 2026. <https://doi.org/10.48550/arxiv.2604.01535>
- [3] Dawei Yan, Haokui Zhang, Guangda Huzhang, Yang Li, Yibo Wang, Qing-Guo Chen, Zhao Xu, Weihua Luo, Ying Li, Wei Dong, Chunhua Shen. "M²: Dual-Memory Augmentation for Long-Horizon Web Agents via Trajectory Summarization and Insight Retrieval." arXiv:2603.00503v1 [cs.CV], 2026. <https://doi.org/10.48550/arxiv.2603.00503>
- [4] Tao Li, Jinlong Hu, Yang Wang, Junfeng Liu, Xuejun Liu. "WEBROUTER: QUERY-SPECIFIC ROUTER VIA VARIATIONAL INFORMATION BOTTLENECK FOR COST-SENSITIVE WEB AGENT." arXiv:2510.11221v1 [cs.CL], 2025. <https://doi.org/10.48550/arxiv.2510.11221>
- [5] Haoyu Liu, Dingcheng Li, Lukas Rutishauser, Zeyu Zheng. "Dual-Modality Multi-Stage Adversarial Safety Training: Robustifying Multimodal Web Agents Against Cross-Modal Attacks." arXiv:2603.04364v1, 2026. <https://doi.org/10.48550/arxiv.2603.04364>
- [6] Yunteng Tan, Zhi Gao, Xinxiao Wu. "Enhancing Web Agents with a Hierarchical Memory Tree." arXiv:2603.07024v1 [cs.AI], 2026. <https://doi.org/10.48550/arxiv.2603.07024>
- [7] Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, Huamin Chen. "Adaptive Vision-Language Model Routing for Computer Use Agents." arXiv:2603.12823v1 [cs.CL], 2026. <https://doi.org/10.48550/arxiv.2603.12823>
- [8] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, Amir Gholami. "Agentic Test-Time Scaling for WebAgents." arXiv:2602.12276v1 [cs.AI], 2026. <https://doi.org/10.48550/arxiv.2602.12276>
- [9] Zikai Xiao, Jianhong Tu, Chuhang Zou, Yuxin Zuo, Zhi Li, Peng Wang, Bowen Yu, Fei Huang, Junyang Lin, Zuozhu Liu. "WebWorld: A Large-Scale World Model for Web Agent Training." arXiv:2602.14721v1 [cs.AI], 2026. <https://doi.org/10.48550/arxiv.2602.14721>
- [10] "ContractSkill: Deterministic Verification and Repair of Multimodal Web Skills." (Authors not fully specified in chunks)
- [11] "OpAgent (Operator Agent)." arXiv:2602.13559v1 [cs.AI], 2026. (Authors not fully specified in chunks)
- [12] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, Yu Su. "SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills." arXiv:2504.07079, 2025. <https://doi.org/10.48550/arxiv.2504.07079>
- [13] Elita A. Lobo, Jingjing Meng, Yang Jiao, Yair Zick, Xu Chen, Nan Xi, Chirag Agarwal, Yan Gao. "STRUCTUREDAGENT: Planning with AND/OR Trees for Long-Horizon Web Tasks." 39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: Efficient Reasoning / arXiv:2603.05294v2 [cs.AI], 2025. <https://doi.org/10.48550/arxiv.2603.05294>

- [14] Yuhan Guo, Cong Guo, Aiwen Sun, Hongliang He, Xinyu Yang, Yue Lu, Yingji Zhang, Xuntao Guo, Dong Zhang, Jianzhuang Liu, Jiang Duan, Yijia Xiao, Liangjian Wen, Hai-Ming Xu, Yong Dai. “WEB-COGREASONER: TOWARDS KNOWLEDGE-INDUCED COGNITIVE REASONING FOR WEB AGENTS.” Preprint. Accepted as a conference paper at ICLR 2026. arXiv:2508.01858v2 [cs.CL], 2026. <https://doi.org/10.48550/arxiv.2508.01858>
- [15] Yuxuan Lu, Jing Huang, Hui Liu, Jiri Gesi, Yan Han, Shihan Fu, Tianqi Zheng, Dakuo Wang. “WEBSERV: A Browser-Server Environment for Efficient Training of Reinforcement Learning-based Web Agents at Scale.” 39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: Multi-Turn Interactions in Large Language Models. arXiv:2510.16252v1 [cs.LG], 2025. <https://doi.org/10.48550/arxiv.2510.16252>
- [16] Wu et al. “Mixture-of-Experts Meets In-Context Reinforcement Learning.” arXiv:2506.05426, 2025. <https://doi.org/10.48550/arxiv.2506.05426>
- [17] Shoubin Yu, Lei Shu, Antoine Yang, Yao Fu, Srinivas Sunkara, Maria Wang, Jindong Chen, Mohit Bansal, Boqing Gong. “Ego2Web: A Web Agent Benchmark Grounded in Egocentric Videos.” arXiv:2603.22529v1 [cs.CV], 2026. <https://doi.org/10.48550/arxiv.2603.22529>
- [18] Ruichen Zhang, Hongyang Du, Dusit Niyato, Jiawen Kang, Zehui Xiong, Ping Zhang, Dong In Kim. “Optimizing Generative AI Networking: A Dual Perspective with Multi-Agent Systems and Mixture of Experts.” arXiv:2405.12472v1 [cs.NI], 2024. <https://doi.org/10.48550/arxiv.2405.12472>
- [19] Sicheng Yang, Yukai Huang, Weitong Cai, Shitong Sun, Fengyi Fang, You He, Yiqiao Xie, Jiankang Deng, Hang Zhang, Jifei Song, Zhensong Zhang. “Egocentric Co-Pilot: Web-Native Smart-Glasses Agents for Assistive Egocentric AI.” ACM Web Conference 2026 (WWW ’26), 2026.
- [20] Abhinav Kumar, Jaechul Roh, Ali Naseh, Amir Houmansadr, Eugene Bagdasarian. “Throttling Web Agents Using Reasoning Gates.” arXiv:2509.01619v1 [cs.AI], September 1, 2025. <https://doi.org/10.48550/arxiv.2509.01619>
- [21] “GUIDE.” (PDF parsing failed; no information available)
- [22] Yaoyao Qian, Yuanli Wang, Jinda Zhang, Yun Zong, Meixu Chen, Hanhan Zhou, Jindan Huang, Yifan Zeng, Xinyu Hu, Chan Hee Song, Danqing Zhang. “WebGraphEval: Multi-Turn Trajectory Evaluation for Web Agents using Graph Representation.” NeurIPS 2025 Workshop: Multi-Turn Interactions in Large Language Models, 2025. <https://doi.org/10.48550/arxiv.2510.19205>
- [23] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, Zhengyang Wu. “WebCanvas: Benchmarking Web Agents in Online Environments.” arXiv:2406.12373v3 [cs.CL], 2024. <https://doi.org/10.48550/arxiv.2406.12373>
- [24] Xiyu Guo, Shan Wang, Chunfang Ji, Xuefeng Zhao, Wenhao Xi, Yaoyao Liu, Qinglan Li, Chao Deng, Junlan Feng. “Towards Generalized Routing: Model and Agent Orchestration for Adaptive and Efficient Inference.” arXiv:2509.07571v2 [cs.MA], 2025. <https://doi.org/10.48550/arxiv.2509.07571>

End of Literature Review Report**Project P79: Cost-Aware Routing for Web Usage Agents****Date:** April 7, 2026**Total Papers Analyzed:** 24 (22 with full analysis, 2 with limited information due to PDF parsing failures)**Total References:** 24